

Zou_lab_control.frontend 中文手册

Jupyter / GUI 实验绘图前端

统一 array 数据契约、live session、fit、selector 与 PDF notes

2026-07-06

Contents

1	写给新读者	1
1.1	frontend 这一层做什么	1
2	qt_fluent 控件库	2
2.1	视觉基调	2
2.2	缩放：一套，不是两套	2
2.3	布局原语（结构上防裁切/重叠）	2
2.4	用原语搭一个对齐的页面	3
3	脉冲编辑器	4
3.1	三个标签页	4
3.2	Edit 页	4
3.3	按字段扫描点工作流	5
3.4	Preview 页	6
3.5	保存与读取：完整的一包（pulse + 预览 + 程序 + 扫描表）	7
4	逐个按钮：完整操作参考（详解）	8
4.1	Edit 页—Channel Names 面板（逐控件详解）	8
4.1.1	这块面板是什么、在哪里	8
4.1.2	顶部信息区：Name / Total / Periods / Visible	8
4.1.3	逐通道两列：左边只读引脚名 + 右边可编辑显示名	10
4.1.4	对齐契约：三列同一行、一起滚动	11
4.2	Edit 页—Delay / Scan 面板与扫描圆点（逐控件详解）	12
4.2.1	顶部信息区：Clock 与 Scan（两个只读显示）	12
4.2.2	顶部入口：Load Array 按钮、文件名与来源开关	12
4.2.3	每个通道一行：延迟输入框（delay field）	14
4.2.4	每个通道一行：单位下拉框与 X 隐藏按钮	14
4.2.5	扫描圆点（scan dot）——把字段变成扫描变量	15
4.3	Edit 页—period 卡片与 DAC 行（逐控件详解）	16
4.3.1	先理解：period 卡片是「时间轴」，不是表格	16
4.3.2	位置标签（卡片顶部）	16

4.3.3	Duration 输入框 (带嵌入式扫描圆点)	16
4.3.4	单位下拉 (Duration 旁边)	18
4.3.5	通道复选框 (每条可见通道一个)	18
4.3.6	DAC (模拟总线) 行—mode 下拉	19
4.3.7	DAC (模拟总线) 行—value 输入框 (带嵌入式扫描圆点)	19
4.4	Edit 页—底部 Control / Connection / Channels 控制条 (逐按钮详解)	20
4.4.1	Connection 卡片——打开后再选连什么	20
4.4.2	Control 卡片: 运行控制按钮 (Stop / On)	20
4.4.3	Control 卡片: 周期编辑按钮 (Remove / Add Period / Add Bracket)	21
4.4.4	Control 卡片: Collapse (折叠左侧面板)	22
4.4.5	Control 卡片: 文件操作 (Save / Load)	22
4.4.6	Channels 卡片: 把隐藏通道调回来 (下拉框 + Add)	22
4.4.7	Channels 卡片: 批量显示/隐藏 (Hide Off / Show All) 与统计标签	23
4.5	Preview 页与扫描高亮在预览里怎么显示 (逐元素详解)	23
4.5.1	它什么时候重画: 没有“刷新”按钮	23
4.5.2	“Show off rows” 开关: 要不要画一直关着的通道	24
4.5.3	状态标签与“Save Figure” 按钮	25
4.5.4	图本身: 数字通道行与模拟 (DAC) 行怎么画	25
4.5.5	repeat (重复): $\times\infty$ 、 $\times N$ 与那个大括号	26
4.5.6	扫描高亮 (橘色透明): 三种被扫描的量怎么标	26
4.6	Scan 页与扫描 workflow (逐元素详解 + 实操配方)	27
4.6.1	顶部: 扫描表列含义说明 (Columns of the scan table)	28
4.6.2	中间: Python 代码编辑器	28
4.6.3	Run 按钮 (绿色)	29
4.6.4	顶部三按钮: Load Program 与两个模板	29
4.6.5	Save Array 按钮	29
4.6.6	下方: 只读扫描表预览	30
4.6.7	完整 workflow: 四步走	30
4.6.8	实操配方一: 一维 duration 扫描	30
4.6.9	实操配方二: 扫一个 DAC 电平 (带符号值 -512..+511, 0 = 0 V)	31
4.6.10	实操配方三: 二维扫描 (duration $\times$ DAC)	31
4.7	Sync 按钮、UNSYNCED 状态与 period 选择	32
5	完整走一遍 (新手教程)	33
5.1	第 0 步: 打开编辑器	33
5.2	第 1 步: 理解 “一段就是一张卡片”	33
5.3	第 2 步: 加 / 删一段, 设时长	33
5.4	第 3 步: 让 channel 在某些段为高	34
5.5	第 4 步: 设延时 (可选)	34
5.6	第 5 步: 在 Preview 看波形	34
5.7	第 6 步: 把一个字段变成扫描参数	34

5.8	第 7 步: 在 Scan 页填扫描表	34
5.9	第 8 步: 运行 / 保存.	34
5.10	完整例子: 扫一个 DAC (DA) 电平.	35
5.11	遇到问题先看哪里.	35
6	Task 控制台 (实验仪表盘)	36
6.1	它是什么.	36
6.2	六种面板类型.	37
6.3	Setting: 每个面板的基本配置	37
6.4	布局: 拖动吸附的模数化卡片	39
6.5	两个常驻标签 + 每面板/每节点的 Edit 标签.	39
6.5.1	从 Add Panel 建节点: 按架构层全暴露	40
6.5.2	Edit 标签 (PanelEditor) 的内容	40
6.6	保存 / 载入布局: 设计一次, 随处载回	41
6.7	Devices 按钮: 只读查看当前设备	42
6.8	启动与接线.	42
6.9	给维护者: 扩展面板	43
7	对外接口完整参考	44
7.1	先读: 密封 API 的设计契约	44
7.2	zf.plot() ——绘图工厂 (最常用)	45
7.3	各图型速查 (plot(kind=...) 的七个面孔)	46
7.4	zf.panel_plot() 与尺寸预设 (仪表盘面板)	46
7.5	zf.show_pulse_gui() ——脉冲编辑器入口	47
7.6	zf.show_task_console() ——Task 控制台入口	47
7.7	SignalHub ——仪表盘数据契约	48
7.8	Measurement 框架——一键可扫描的测量 (Logic 节点)	48
7.9	PDF 渲染 API	49
7.10	一页速查 (“我想做 X -> 调用 Y”)	50
8	绘图 API.	51
8.1	统一的数组契约.	51
8.2	直方图与阈值.	51
8.3	实时刷新.	51
9	保存与回看图 (notebook 与 GUI 同一套核心)	54
9.1	一套核心, 两种调用	54
9.2	存一张图: p.save().	54
9.3	富捕获: bind_source 把“数据从哪来”盖进去	55
9.4	回看: na.load_figure 与 exp.figure_viewer	55
9.5	面板 Save 与 pulse 预览也走同一套	56
10	PDF 渲染 API	57
10.1	render_tex_pdf	57
10.2	notes 风格的整本手册	57

11	修改指南	58
11.1	我要加一个新 Fluent 控件	58
11.2	我要给脉冲 GUI 加一行	58
11.3	我要改截图 QA.	58
11.4	我要改本手册.	59

1

写给新读者

1.1 frontend 这一层做什么

`Zou_lab_control.frontend` 是独立的用户界面层，拥有：绘图、实时刷新、Fluent/PyQt 控件、脉冲编辑器、以及 PDF/手册渲染。它**不**拥有任何硬件动作——脉冲编辑器只是 `PulseTableState` 加一个传入 `sequencer` 的前端。

主要文件：

`qt_fluent.py` 可复用的 Fluent/Confocal 风格与控件，以及结构上防止裁切/重叠的布局原语。

`pulse_gui.py` Confocal 风格脉冲编辑器，编辑 `PulseTableState` 并调用传入的 `SequencerDevice`。

`live.py, data_figure.py, canvas.py` 绘图与实时刷新层。

`notes.py` PDF/手册渲染 (`render_tex_pdf` 等)。

一条总规则

PyQt 前端应复用 `qt_fluent.py`，不要在每个 GUI 里手写一套 Qt 样式。布局应使用本章介绍的原语，而不是逐行硬编码像素几何——这是“对齐契约”能在不同 Windows DPI 下稳定成立的原因。

2

qt_fluent 控件库

2.1 视觉基调

统一用 Segoe UI、12 pt、背景 `##F3F3F3`、文字 `##323130`、强调色 `##77AADD`、圆角 4 px、扁平 1 px 卡片边框。`FluentGroupBox` 是带灰色标题胶囊的白色卡片，用一条 1 px `DIVIDER` 细边框勾勒（不是软阴影——软阴影每帧都要把整张卡重新栅格化再高斯模糊，35 站点大网格卡冷载入实测多花 250 ms，扁平边框零开销且同样清晰地界定卡片边界）；这条边框是密封的构造细节，不要把它暴露成按调用可配的开关，也不要把它换成普通表格。用 `FluentComboBox/FluentSpinBox/FluentScrollArea` 让弹出列表、计数器、细滚动条共享同一套样式。

2.2 缩放：一套，不是两套

所有固定几何用 `set_fluent_scale()` / `scaled_px()`。不要在某个 GUI 模块里再造第二套缩放系统。

2.3 布局原语（结构上防裁切/重叠）

下面这些是新加入的原语。优先用它们，而不是逐行 `setFixedGeometry`：

Metrics 缩放后的间距/尺寸 token: `margin/gap_row/gap_item/gap_tight/row_h/dot`。
在使用时再读取，它们随当前 DPI 缩放变化。

measure_text_width(texts, ...) 按当前缩放返回能容下最宽文本的标签列宽。让标签列宽“由内容驱动”，避免写死宽度导致 `cooling_pgc`、`da_dipole` 这类名字被截断。

ElidedLabel 文本过长时用 ... 省略，并把完整文本放进 tooltip。

FluentScanDot 字段旁的圆形开关：未绑定时是空心灰点；绑定后是实心橙点，中间显示它的 1-based slot 编号。

mark_scan_field(widget, bound=...) 给绑定到 slot 的字段套上“橙色 + 禁用”外观。

FluentLabeledField / FluentFormGrid label : widget 行，标签列共享、对齐，每行同高。
把一摞表单堆叠起来时天然对齐，不需要逐行写死几何。



Figure 2.1: 标签列对齐 + 扫描圆点的一行表单。绑定后字段变橙并禁用。

2.4 用原语搭一个对齐的页面

一个稳健、对齐的 Fluent 页面的骨架：

Python

```
from Zou_lab_control.frontend import qt_fluent as qf

label_w = qf.measure_text_width(["Clock:", "Delay:", "Total:"]) #
↳ 由内容决定列宽
form = qf.FluentFormGrid(label_width=label_w)
form.add_row("Clock:", qf.FluentLineEdit("50 MHz"))
form.add_row("Total:", qf.FluentLineEdit("1.2 ms"))
# 每行同高 (Metrics.row_h()), 标签列共享 label_w; 堆叠时自动对齐。
```


3

脉冲编辑器

3.1 三个标签页

PulseSequenceEditor 有 **Edit / Preview / Scan** 三个标签页。

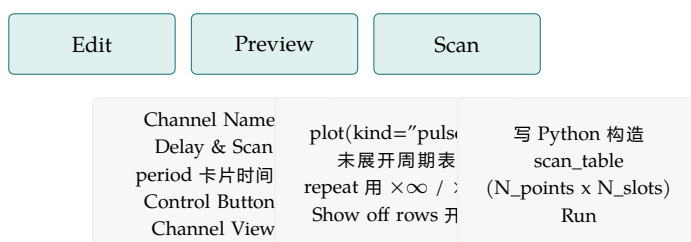


Figure 3.1: 编辑器三页: Edit 编脉冲表, Preview 画未展开的时序图, Scan 写代码生成扫描表。

3.2 Edit 页

按区域理解:

Channel Names and Duration 脉冲名、总时长、period 数、可见数。左侧 raw 列在能读到 XDC 时显示物理 package pin (如 M17/M13); chNN 保存在 tooltip 和 JSON/API state。右侧是可选的显示标签。

Delay \& Scan Clock (只读显示当前 sequencer 时钟, 如 50 MHz · 20 ns, 由硬件固定、不可编辑)、每条可见 channel 的固定延时、以及 Load Array (载入扫描表文件, 旁边显示已载入文件名) 和一个来源开关 (关 = 用 Scan 页生成的扫描表; 开 = 用载入文件的数组)。Scan 页本身从顶部的 Scan 标签直接进入。

period 卡片时间线 每张卡是一段持续时间, 不是网格表。卡片含 duration、单位选择、位置 (Period 1/5) 和每条 channel 的勾选框。

Control Buttons On Pulse/Stop Pulse、增删 period、repeat bracket、保存/读取、Sync。On Pulse 先 prepare/上传当前脉冲再 start; Sync 把设备上实际应用的脉冲拉回编辑器 (用于 notebook/API 改了设备而 GUI 没跟上时)。

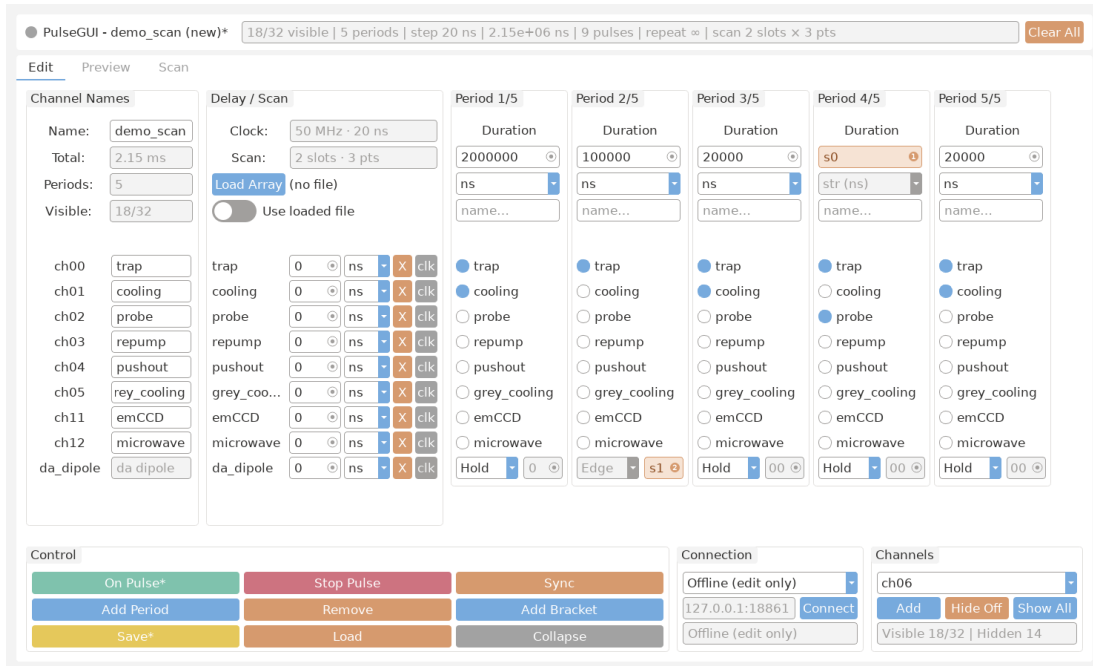


Figure 3.2: Edit 标签页的真实截图（脱机离屏渲染）。左侧 Channel Names 与 Delay / Scan 列，中间是横向滚动的 period 卡片时间线，下方是 Control / Channels 控制条。橙色字段是已绑定到扫描 slot 的字段，字段右侧的圆点显示它的 slot 编号。

Channel View 从隐藏列表加回 channel、隐藏全程 off 的 channel、显示全部。

对齐契约

channel 名列、延时列、period 勾选框列共享**同一个**竖直滚动位置。period 卡片可以横向滚动（当作时间线），但它们**不能**有各自独立的竖直滚动条。同一条 channel 的 raw 标签、延时框、第一个 period 勾选框，三者的 `mapTo(editor, QPoint(0,0)).y()` 必须相等。

3.3 按字段扫描点 workflow

扫描以**命名 slot**为单位 (`s0`, `s1`, ...), 没有 x/y。只有 **duration** 和 **DAC 值** 两类字段可扫, channel 延时**不可扫**。

1. 在 Edit 页点某个 duration / DAC 字段旁的**扫描圆点**。该字段变橙、显示它的 slot 编号；这等于 API 的 `state.bind_field(kind, target)`。
2. 打开 **Scan** 页。页面顶部三个按钮可以一键填充编辑器：Load Program (见下)、Template: column_stack (每个 slot 一列，**默认**模板)、Template: grid (两个数组的外积网格)。两个模板都**按当前 slot 数自动适配**；默认显示 column_stack 模板。也可以直接写 Python，把一个 `N_points x N_slots` 数组赋给变量 `scan_table` (命名空间里有 `np`、`math`、`n_slots`)。
3. Load Program 接受**保存/笔记本产生的所有格式**，按后缀自动识别：`.py/.txt` 是生成扫描表的 Python 程序（载入代码框）；`.npy/.csv` 是扫描数组（直接装入“载入文件”来源，等同 Load Array）；`.json` 既可以是**保存的脉冲**（取其内嵌扫描表），也可以是**保存的编译程序** `< 名字 >_program.json`——后者存的是硬件线上的原始值，载入时会**自动换算回用户单位**（duration 节拍数 $\times$ 节拍长 $\rightarrow$ ns；DAC offset-binary 码 $-512 \rightarrow$ 带符号值），再照常 snap 到当前绑定的 slot。装数组/表需要先绑定至少一个扫描槽。

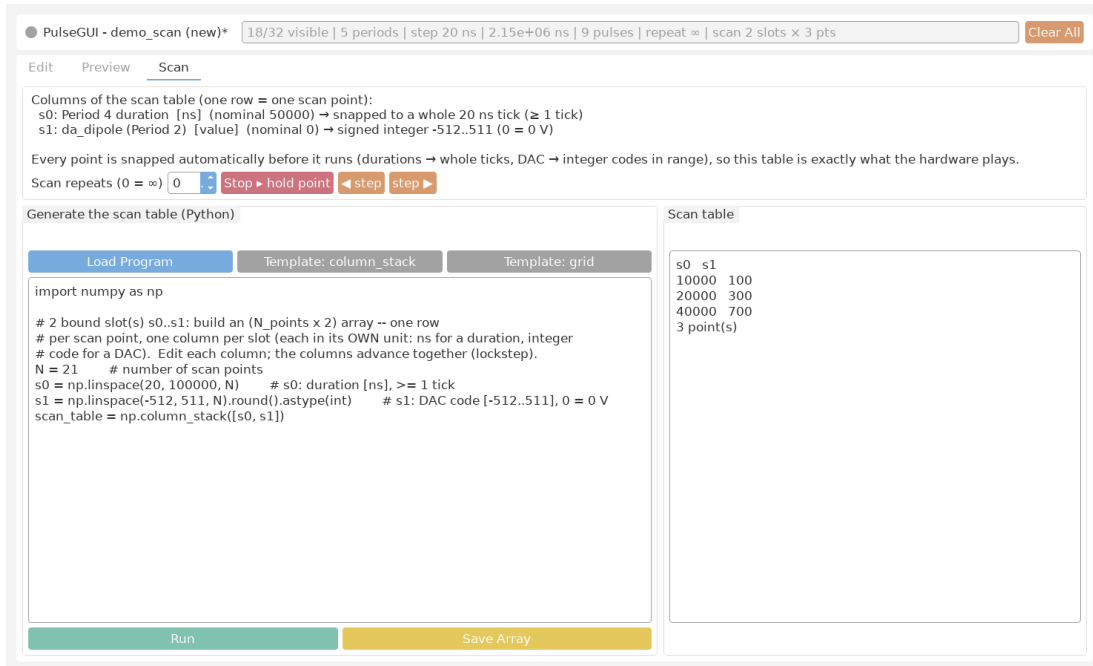


Figure 3.3: Scan 标签页的真实截图。顶部列出扫描表的各列（每个绑定的 slot 一列，含它绑的字段与单位）；代码框上方是 Load Program（接受.py 程序、.numpy/.csv 数组、保存的脉冲或编译程序.json）与两个模板按钮，下方是 Run / Save Array 按钮；右侧是只读的扫描表预览。

- 点 **Run** 生成扫描表（写回“生成”来源）；要用现成数组文件就去 Delay / Scan 面板点 **Load Array**（写“载入”来源）。**Run 写“生成”、Load Array 写“载入”**；用 Delay / Scan 面板里的来源开关在两者之间切换。

自动 snap：你看到的就是硬件跑的

无论用 Run、Load Array 还是 API，扫描表在背后都会被**自动 snap 成硬件能跑的合法值**：**duration** 列对齐到**整数个节拍**（且至少 1 个节拍，不会塌成 0），**DAC** 列四舍五入成**带符号整数**并夹到该总线的带符号范围（10-bit 即 **-512 .. +511**，**0 = 真零点 0 V**；硬件线上为 offset-binary 码 = 值 + 512，由编译器自动换算）。Scan 页信息栏会逐列写明每个 slot 的允许取值与 snap 规则，所以预览/保存的表与实际下发的完全一致。

Scan 页里的代码（column_stack 默认模板，2 个 slot 时）

```
import numpy as np
# 2 个已绑定 slot: s0、s1。行 = 一个扫描点，列 = slot (ns / DAC code)。
points = np.arange(20, 200e3, 20)          # 基准扫描
s0 = points
s1 = 200e3 - s0                             # 反相关: s0 + s1 = 200 us 恒定
scan_table = np.column_stack([s0, s1])
```

3.4 Preview 页

Preview 复用 `frontend.plot(..., kind="pulse")`，在打开标签页或切换 **Show off rows** 时自动重画，**没有手动 refresh 按钮**。它画**未展开**的周期表；y 轴标签是 channel 显示标签（不显示 **Pulse** 标题）；repeat 用 $\times\infty$ / $\times N$ （绝不出现字面 **inf**）；绑定 slot 的区段用贯穿所有 channel 的半透明标记画出；模拟总线行画成空心阶梯线。

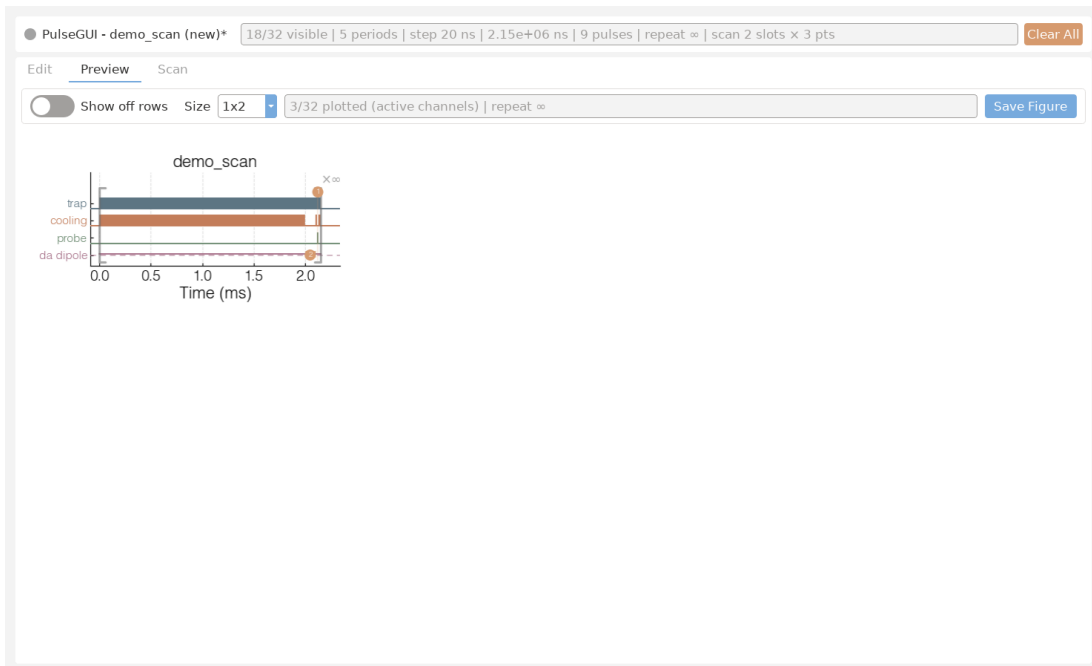


Figure 3.4: Preview 标签页的真实截图。画的是**未展开**的周期表；每条数字 channel 一行方波，模拟总线画成空心阶梯线。绑定 slot 的区段用贯穿所有行的半透明色带标出：duration 的标号在顶部、DAC 的在底部、delay 的在对应通道行，并带白色描边避免压字。

Show off rows 同样管 DAC 行：开关**关**时，预览只画有信号的行——一条全程为 0 / hold 的**空闲 DAC 总线会被隐藏**（就像一条全程 off 的数字通道），只有带非零值或被扫描的 DAC 才显示；开关**开**时，所有 DAC 总线都显示。无论隐藏与否，DAC 的位通道**绝不会**作为单独的数字行泄漏出来。

3.5 保存与读取：完整的一包（pulse + 预览 + 程序 + 扫描表）

Save 一次写一整包，四件文件落在同一目录、共享同一个文件名前缀：

- < 名字 >.json ——脉冲本体（编辑器的完整状态，load 回来逐项还原）；
- < 名字 >.png ——预览图（和 Preview 页看到的一样）；
- < 名字 >_program.json ——**编译好的、FPGA 可直接上传的程序**（与点 On Pulse 实际上传的内容一致；**无论是否扫描都会保存**：有扫描槽和扫描表时是扫描程序，否则是普通运行程序）；
- < 名字 >_scan.npy ——（仅当存在扫描表时）扫描表原始数组。

某一件写失败不会让整个保存失败：状态栏会写明哪些写成功、哪些被跳过及原因。

Load 把整包读回来：在 Edit 页点读取并选 < 名字 >.json，编辑器还原脉冲本体；若同目录有配套的 < 名字 >_scan.npy，它会被**自动重新挂回**“载入文件”来源（Delay / Scan 面板会显示该文件名），内嵌在.json 里的扫描表则照常成为“生成”来源——两者在保存时刻数值相同，来源开关保持在“生成”。**误选了 _program.json 或 _scan.npy?** 只要同目录有对应的脉冲 .json，软件会自动改为加载脉冲本体并提示一句。 .png 与 _program.json 是衍生物，load 后由当前状态随时重新生成，无需单独读取。

4

逐个按钮：完整操作参考（详解）

本章面向**第一次打开本软件的人**，把脉冲编辑器 Edit / Preview / Scan 三个页面上的**每一个控件**逐个讲清：作用、能填什么、点击或输入后会发生什么、常见坑、以及一个实用提示。配合每节的真实截图阅读。

4.1 Edit 页—Channel Names 面板（逐控件详解）

这一节专为**第一次打开本软件、从未见过这个界面的人**写。我们会把 Edit（编辑）页最左边那一块叫做 **Channel Names（通道名字）** 的白色卡片，从上到下、一个控件接一个控件地讲清楚：它是干什么的、能填什么、点了或填了之后会发生什么、新手最容易踩的坑、以及一个能立刻用上的小技巧。读完这一节，你应该能独立看懂这块面板上的每一个字、每一个输入框。

4.1.1 这块面板是什么、在哪里

Channel Names 面板是 Edit 页**最左边**的一张白色、带标题的卡片，标题就写着 **Channel Names**。它的职责只有两件事：一是显示并让你修改整条脉冲序列的**身份信息**（名字、总时长、周期数、可见通道数），二是为每一条**通道**提供两个名字——一个是硬件上固定的物理引脚名（只读），一个是你自己起的、好记的显示名（可改）。

先记住一句话

这块面板里能真正**改变硬件行为**的东西非常少。除了最上面的 **Name** 是给整个文件起名，下面每条通道右边的“显示名”**只改怎么显示、不改硬件**。其余字段全部是**只读**的、自动算出来的，看就行，改不了也不用改。

4.1.2 顶部信息区：Name / Total / Periods / Visible

面板顶部是四行，每行左边是一个标签，右边是对应的值。它们一行一个字段，从上到下排列。除了 **Name** 可以编辑，其余三个都是**只读**的——也就是说你点进去也敲不动，它们的数值会随着你在别处的修改**自动实时更新**。

Name（名字） 这是**唯一可编辑**的顶部字段，是一个普通的文本输入框，里面装着这条脉冲序列的名字。

- **作用**：这个名字有两个真实用途——保存时它会成为输出的 **.json** 文件名；画预览图时它会

Channel Names

Name:

demo_scan

Total:

2.15 ms

Periods:

5

Visible:

18/32

ch00

trap

ch01

cooling

ch02

probe

ch03

repump

ch04

pushout

ch05

rey_cooling

ch11

emCCD

ch12

microwave

da_dipole

da dipole

成为图的标题。所以起一个有意义的名字，以后翻文件、看图都省事。

- **接受什么**：任意文本。建议只用字母、数字、下划线，避免空格和 `/`、`\`、`textbackslash`、`:` 这类在文件名里非法或容易出问题的字符。
- **填了会怎样**：你敲进去的文字会被记住，下次保存就用它当文件名，下次画图就用它当标题。
- **常见坑**：如果名字里带了路径分隔符或非法字符，保存成文件这一步可能会出错或生成奇怪的文件名；两条序列起了同一个名字，保存时后者会覆盖前者的同名文件。
- **小技巧**：把关键参数编进名字里，比如 `cooling\_scan\_2150us`，这样不打开文件、光看文件名就知道它是干嘛的。

Total (总时长) **只读**。显示整条序列的总持续时间，带单位 `ns` (纳秒)，例如 `2150000 ns`。

- **作用**：让你一眼看到这条序列从头到尾跑多久。它是把**所有周期的时长加起来**得到的。
- **怎么变**：你改不了这个框本身。但只要你在别处改了任何一个周期的时长，这里的 Total 会**立刻重新计算并刷新**，不需要你手动按任何按钮。
- **常见坑**：新手常以为可以直接在这里打一个目标总时长——不行，它是算出来的结果，不是输入。想改总时长，得去改各个周期的时长。
- **小技巧**：把它当成一个**校验仪表**。如果你心里预期总长是 2.15 ms，而这里显示的 `ns` 数对不上，那多半是某个周期时长填错了，回去逐个核对。

Periods (周期数) **只读**。显示这条序列里**周期**的数量，例如 `5`。

- **作用**：告诉你这条序列被切成了几个时间段 (周期)。每个周期是序列在时间轴上的一段，有自己的时长和各通道的开关状态。
- **怎么变**：同样是自动的。你在别处增加或删除周期，这个数字会跟着加减。
- **常见坑**：它和 **Visible** 是两回事——**Periods** 数的是**时间上的段数**，**Visible** 数的是**通道 (行) 的条数**，别混淆。
- **小技巧**：当预览图看起来段数不对时，先瞄一眼这个数字，能快速确认你到底建了几个周期。

Visible (可见通道数) **只读**。以 **已显示/总数** 的形式显示通道计数，例如 `18/32`。

- **作用**：斜杠左边是当前**正在显示**的通道数，右边是**全部**通道数。`18/32` 表示一共有 32 条通道，目前面板上显示了其中 18 条。
- **为什么会少显示**：硬件可能有很多条通道，但你这条实验通常只关心一部分；界面会把无关的通道隐藏起来，让你专注在用到的那些上。
- **常见坑**：如果你觉得“某条通道怎么找不到了”，先看这里——如果左边数字小于右边，说明确实有通道被隐藏了，它不是消失了，只是没显示。
- **小技巧**：把它当作一个**完整性检查**。右边的总数应当等于你硬件实际的通道总数；左边的数应当等于下面逐通道列表里你能数出来的行数。

4.1.3 逐通道两列：左边只读引脚名 + 右边可编辑显示名

顶部信息区下面，是**一条通道一行**的列表，只列出当前可见的那些通道 (也就是 **Visible** 左边那个数那么多行)。每一行有两列：左边是硬件给定的、改不了的物理名字；右边是一个你可以随便改的、好记的显示名输入框。

左列：raw 标签 (物理引脚名，只读) 每行最左边是一个**只读**标签，显示这条通道对应的**物理封装引脚名**。

- **它显示什么**：如果软件知道这条通道接到了板子上的哪个引脚（来自板卡的 XDC 约束文件），就显示那个引脚名，例如 `M17`；如果不知道，就退而显示内部名 `chNN`，例如 `ch00`。
- **为什么只读**：这是硬件接线决定的事实，不是你能在软件里改的。它代表“第几号物理输出”，改它没有意义，所以软件不让你动。
- **名字太长会怎样**：当引脚名太长、一行放不下时，界面会用 `...` 把它截断显示；**完整文本会放进鼠标悬停提示 (tooltip) 里**。
- **一个永远的保底**：不管左边显示的是引脚名还是被截断的文字，**完整的内部名 `chNN` 始终能在该标签的 tooltip 里看到**。所以你随时可以把鼠标停在上面，确认这到底是第几号通道。
- **常见坑**：不要把这个只读的 `M17` 当成可以重命名的东西去双击——它不会进入编辑状态。想起别名，请用它右边那个框。
- **小技巧**：当你需要把界面上的某条通道和实际接线、和 FPGA 比特位对上号时，悬停看 tooltip 里的 `chNN`，这是最可靠的对照方式。

右列：display-label 输入框 (显示名, 可编辑) 每行右边是一个**可编辑**的文本框，让你给这条通道起一个人类好记的名字，例如 `cooling` 或 `da\_dipole[0]`。

- **作用**：纯粹是**显示用**的别名。你填的名字会出现在界面的各处标签上，以及预览图的 y 轴上，让你看图、找通道时一眼认出谁是谁。
- **关键事实——它不改硬件**：这个显示名**只影响“怎么显示”，绝不改变硬件的比特位顺序**。也就是说，无论你把第 0 号通道叫 `cooling` 还是叫 `laser\_A`，它在 FPGA 里仍然是同一根输出、同一个比特位。改名只换标签，不换接线。
- **接受什么**：任意文本，可以带方括号和下标风格的写法，例如 `da\_dipole[0]`，方便你给同类通道编号。
- **填了会怎样**：你敲完之后，界面上引用这条通道的地方、以及预览图 y 轴上对应的那一行，都会换成你起的新名字。
- **常见坑**：千万别误以为“把两条通道的显示名对调，就能交换它们的硬件输出”——不会的，硬件顺序由比特位决定，显示名只是贴在外面的标签。还有，给两条通道起完全一样的显示名虽然不报错，但看图时容易把自己搞糊涂。
- **小技巧**：用统一的命名规则（比如冷却光统一叫 `cooling`、偶极阱用 `da\_dipole[0]`、`da\_dipole[1]`），预览图 y 轴会立刻变得清爽好读，排查实验问题时半功倍。

4.1.4 对齐契约：三列同一行、一起滚动

这块面板看起来简单，但它和它**右边那块**面板之间有一个重要的、刻意保证的**对齐约定**，新手了解一下能少很多困惑。

同一条通道，三样东西在同一行 对任意一条通道来说，它在本面板里的**左列只读引脚名**、它在右边相邻面板里的**delay (延时) 框**、以及它在周期区里的**第一个周期的勾选框**，三者**处在同一行上**——它们的**垂直中心是对齐的**。

三列一起上下滚动 这三列在垂直方向上是**联动滚动**的：你往下滚动通道列表时，引脚名、delay 框、勾选框会**一起同步移动**，绝不会出现“名字滚到下面去了、它的延时框却还停在上面”这种错位。

这个对齐为什么重要

因为你经常需要**横着读一整行**：先在最左边认出这是哪条通道（看引脚名或显示名），再顺着同一行往右看它的延时是多少、它在第一个周期里开没开。正因为三列严格对齐、一起滚动，你才可以放心地“一

行从左读到右”，不用担心读串行。如果哪天你觉得名字和延时框对不上，那通常意味着界面出了渲染问题，而不是这条通道的设定真的错位了。

4.2 Edit 页—Delay / Scan 面板与扫描圆点 (逐控件详解)

本节面向**从未见过本软件界面**的新用户，逐个控件讲解 Edit (编辑) 页中第二个面板——**Delay / Scan (延迟 / 扫描) 面板**。它是从左数第二张白色带标题的卡片，标题写着 **Delay / Scan**。这张卡片管两件事：一是给每个通道设置**固定的延迟 (delay)** (每条 channel 一个固定值，**不是扫描量**，用来补偿线缆和设备的固有时延)；二是顶部的扫描入口 (**Load Array** 按钮 + 来源开关)，用来给在别处 (duration / DAC 字段) 绑定的扫描槽**填数值**。下面先看顶部的一组全局控件，再看每个通道一行里的各个控件，最后专门讲贯穿全软件的**扫描圆点 (scan dot)**——它只出现在 duration 和 DAC 字段上，**不在**这里的延迟字段上。

4.2.1 顶部信息区：Clock 与 Scan (两个只读显示)

面板顶部有两条**只读**信息。只读的意思是它们只用来**显示状态**，你无法直接在上面打字修改；它们的内容会随着你在别处的操作自动更新。

Clock 作用：**只读**显示当前测序器 (sequencer) 的**时钟频率与节拍长度**，形如 **50 MHz · 20 ns**——前者是硬件时钟，后者是一个节拍 (tick) 的时间，也是硬件最小的时间分辨率。它**由硬件固定、不可编辑** (所以这里是一个灰色只读框：节拍不是用户能改的量，改它没有意义)。关键影响：你在本软件里输入的**任何时间量** (周期时长、通道延迟、扫描表里的 duration) 都会被自动**量化**成这个节拍的整数倍。例如节拍是 **20 ns** 时你填 **30 ns**，会被对齐到最接近的倍数 (**20 或 40 ns**)。常见的坑：新手会奇怪“为什么我填的数被改了”——这正是量化在起作用，不是 bug。实用提示：先把目标时间换算成节拍的整数倍再填，所见即所得。

Scan 作用：用一行简短文字**汇总**当前已经绑定的扫描槽 (slot) 数量和扫描点数，形如 **3 slots · 3 pts**，意思是“当前绑定了 3 个扫描槽、扫描表里有 3 个扫描点”。它接受的同样不是输入，而是自动统计。点击或输入：你不能在这里点或改，它只随你在各处绑定/解绑扫描圆点、或载入扫描表而变化。常见的坑：当这里显示 **0 slots** 时，说明你还没有绑定任何扫描变量，这次运行就是一次普通的单点序列，而不是扫描。实用提示：每次绑定完圆点后瞄一眼这里，确认 slots 数和 pts 数都符合你的预期，是排查“为什么没在扫”的最快办法。

4.2.2 顶部入口：Load Array 按钮、文件名与来源开关

Load Array 按钮 作用：打开一个**文件选择对话框**，从磁盘载入一份**扫描表 (scan table)** 文件，存为“载入”来源。可接受的文件类型是 **.npy / .csv / .txt / .json**。文件的格式约定非常重要：**每一行代表一个扫描点，每一列代表一个已绑定的扫描槽**。点击后会发生什么：弹出系统文件对话框，你选中文件后，表里的数值被读进来并**自动 snap** (duration 对齐节拍、DAC 夹到带符号范围 **-512 .. +511**)，同时**右边的标签显示该文件路径的末尾十几个字符**，并把来源开关**自动打开** (即开始使用这份载入数组)。常见的坑：行列别搞反 (行 = 点、列 = 槽)；列数和槽数对不上时会被截断/补零。实用提示：不想手写文件就用 Scan 页的代码生成；Load Array 更适合从外部仪器或脚本导出的现成数据。

已载入文件名标签 作用：显示**当前载入**的扫描表文件路径 (从左侧省略，保留**末尾十几个字符**，鼠标悬停看完整路径)。没有载入任何文件时显示 (**no file**)。它是只读的，仅用于让你确认“开关打开时用的是哪份文件”。

来源开关 (Use loaded file) 作用：一个**切换开关**，决定本次运行用哪份扫描表。**关** (默认) = 用 Scan 页代码 **Run** 出来的“生成”扫描表；**开** = 用 **Load Array** 载入的那份“载入”数组。点击后会发

Delay / Scan

Clock:

50 MHz · 20 ns

Scan:

2 slots · 3 pts

Load Array

(no file)

Use loaded file

trap	0	ns	X	clk
cooling	0	ns	X	clk
probe	0	ns	X	clk
repump	0	ns	X	clk
pushout	0	ns	X	clk
grey_coo...	0	ns	X	clk
emCCD	0	ns	X	clk
microwave	0	ns	X	clk
da_dipole	0	ns	X	clk

Figure 4.2: Delay / Scan 面板的整体外观。顶部是 Clock / Scan 只读信息、Load Array 按钮（旁边显示已载入文件名）和一个来源开关；下方每个通道一行，依次是延迟输入框（数字校验、超界自动夹回）、单位下拉框和一个 X 隐藏按钮。

生什么：立即切换活动来源并重画预览/汇总，**不需要重跑代码**。常见的坑：开关打开但还没载入任何文件时，会提示你先用 Load Array，并自动回到“生成”来源。实用提示：把同一组绑定下“代码生成”和“外部文件”两套扫描表都准备好，用这个开关一键对比，省去反复 Run/Load。

去哪写扫描代码 点界面顶部的 Scan 标签即进入 Scan 页写代码（见“按字段扫描点 workflow”一节）。

4.2.3 每个通道一行：延迟输入框 (delay field)

面板下方是**逐通道**的若干行，每个通道占一行。每行最主要的控件是**延迟输入框**，它是一个单行文本框，默认显示 0。

延迟输入框 作用：给**这一个通道的所有边沿**加上一个**固定的时间偏移**，用来补偿线缆长度、设备响应等带来的固有时延。它是一个**固定值**，**不是扫描量**——延迟字段没有扫描圆点（要扫的是 duration 或 DAC 值，不是延迟）。可接受的值：一个时间数值，**可正、可负、可零**——正值把该通道的所有上升/下降沿整体**往后推**，负值整体**往前推**，零表示不延迟。**范围很大**：延时由一个加在输出上的**硬件事件调度器**实现，大小由一个 32 位字段决定（约 ± 42.9 s，毫秒级 emCCD 延时直接可用），所以 $|delay|$ 不能超过这个上限。**超界会被自动夹回**该上限（编辑完成时自动发生），输入框的悬浮提示也写明了当前上限；并且该框**只接受数字、小数点、e 记数法与正负号**，其它字符在键入时直接被拒绝。它**不会改变任何周期 (period) 的时长**，只是把该通道的输出整体平移——该通道在 fire 之后**安静**到延迟时间才开始，**第一帧是真实的**（不是预览里那种循环绕回的样子）。输入后会发生什么：该数值先被**量化**成节拍的整数倍，再把该通道在时间轴上的所有跳变按这个偏移挪动。常见的坑：(1) 以为延迟会拉长或缩短脉冲——不会，它只平移边沿，脉冲宽度和周期长度不变；(2) 忘了它是**按通道**独立的，改了 A 通道不影响 B 通道，也**绝不**打扰别的通道的时序；(3) 负延时等价于给**其它所有通道**各加一个正延时（编译器用一个全局平移 G 实现），所以一组延时的**跨度**（最大减最小）也要落在 32 位范围内；真正的资源约束是**在途翻转数**（每个信号一个有限深度的事件 FIFO），超出时编译会报清楚的错。实用提示：先用示波器量出某条线相对基准慢了多少 ns，把这个值（取正补偿滞后）填进去即可；负延迟同样合法，适合让某通道**提前**触发。模拟总线 (DAC) 的值**也能**用同样的方式延时（同一个 32 位范围、同一套事件调度器），且一条 DAC 总线的 10 个 bit **共享一个**延时。

4.2.4 每个通道一行：单位下拉框与 X 隐藏按钮

单位下拉框 (ns / us) 作用：选择该行延迟数值的时间单位。延迟的可选单位**只有 ns 和 us 两个**——延迟被延时线深度限制在约 $\pm 40 \mu\text{s}$ 以内，ms/s 一定超界，而 str (ns)（仿射表达式单位）对一个固定、不可扫描的延迟也没有意义，所以这里不再提供。点击后会发生什么：下拉展开，选中某个单位后，左边输入框里的数字就按这个单位解释（例如填 1 配 us 等于 1 微秒）。常见的坑：单位选错会让延迟差出 1000 倍（把 us 当成 ns）；另外最终值仍会被时钟节拍量化，所以填一个比节拍还小的组合可能被对齐成 0。实用提示：日常多用 ns，较长的固定延迟用 us。

X 按钮 (隐藏通道) 作用：把该通道从表里**隐藏**，让界面更清爽。点击后会发生什么：这一行从 Delay / Scan 表里消失，该通道被移到**隐藏列表**中。它去哪了：底部的 Channels (通道) 卡片会记录被隐藏的通道，你可以从那里把它**重新调出来**。常见的坑：以为 X 是“删除/禁用通道”——它只是**视觉上隐藏**，不会改动该通道的脉冲内容；找不回来时别新建，去底部 Channels 卡找。实用提示：把当前实验用不到的通道隐藏起来，可以让一长串通道里真正关心的几行更醒目、对齐更易读。

clk 按钮 (把通道接到 FPGA 时钟) 作用：把**这一条通道的输出引脚**接到 **FPGA 时钟**，用来给外部设备（比如 DAC）提供锁存选通。点击后会发生什么：该通道被标记为 clk 通道——它**从脉冲引擎里移除**（不再由边沿表驱动，所以引擎**绝不会覆盖**它的 clk 行为），它这一行的**延迟框和单位框都变灰禁止编辑**（对一条 clk 通道没有意义），并且从 **Preview 里隐去**（它输出的是时钟方波，不是脉冲）。再点一次恢复成普通的引擎驱动通道。实现上，编译时把该通道的 mask 位强制清 0，并把一个 62

位的 `clk_enable` 掩码随程序下发; FPGA 顶层用 `out_final[n] = clk_en[n] ? ~clk : out[n]` 把反相的 `clk` 接到该引脚 (运行时由 CTRL 寄存器选择, 换 `clk` 通道不需要重新综合, 但首次启用这套 mux 需要一次重新构建 bitstream)。为什么是反相 `clk`: DAC 的并行数据位是在 `clk` 上升沿改变的, 若选通也用上升沿, DAC 会在数据正在跳变的瞬间锁存而锁进错误的中间码 (表现为“两个 DA edge 段之间偶发出现第三种值”); 用反相 `clk` 让 DAC 改在下降沿 (数据眼图正中、最稳) 锁存, 从根本上消除这个竞争。常见的坑: DAC 总线 (`da_dipole` 等) 的成员位不能单独设成 `clk` (会破坏 DAC), 所以这些行的 `clk` 按钮是禁用的。实用提示: 要给外部 DAC 喂时钟, 就把对应那条通道 (如某个 `da_clk` 线) 设成 `clk`, 不用再在脉冲里手动翻转它。

4.2.5 扫描圆点 (scan dot) ——把字段变成扫描变量

扫描圆点是贯穿整个软件的关键控件, 它出现在可扫描的两类字段上: 周期的时长 (duration) 字段和 DAC 值字段。延迟 (delay) 字段没有扫描圆点——延迟是每条 channel 的一个固定值, 不能被扫描 (在 API 里 `state.bind_field("delay", ...)` 会直接报错)。扫描圆点嵌在输入框的右边缘, 外形和行为都像 spinbox (数值微调框) 那个上下小按钮, 垂直居中。它的核心作用, 是把一个普通的数值字段“升级”为一个会在实验里被自动扫过的扫描槽 (slot)。

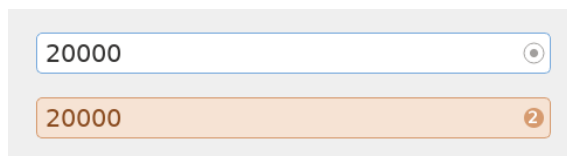


Figure 4.3: 扫描圆点的两种状态。左: 未绑定——空心灰色圆环加一个小小的中心点; 右: 已绑定——实心橙色圆点, 里面显示该槽的 1 起编号 (`s0` 显示 “1”), 同时字段变成淡橙底、深橙字的只读样式。

未绑定时的外观 一个空心灰色圆环, 中间有一个很小的中心点。看到这个样子, 说明该字段还是普通可编辑的数值, 没有参与扫描。

点击一次: 绑定 (bind) 作用: 把该字段绑定到一个扫描槽。点击后会发生三件事: (1) 字段变成橙色样式——淡橙色背景、深橙色文字, 并变为只读 (你不能再直接在框里打字, 因为它的值现在由扫描表决定); (2) 圆点变成一个实心橙色圆点, 里面显示该槽的从 1 起的编号——槽 `s0` 显示 “1”, `s1` 显示 “2”, 以此类推; (3) 字段里原本存的数值被改写成一个符号表达式 `s0 / s1 .....`, 表示“此处的值取自对应扫描槽”。

再点一次: 解绑 (unbind) 作用: 把字段从扫描槽上解开, 恢复成普通可编辑数值。贴心之处: 当你再次绑定同一个字段时, 之前为它在扫描表里填过的那些数值会被记住并恢复, 不用重填。

这等价于哪条 API 这个圆点就是图形界面里对应的一次 API 调用 `state.bind_field(kind, target)`——也就是说, 鼠标点圆点和在代码里调用 `bind_field` 是同一件事的两种入口。

编号与重新编号 (重要的坑) 槽按绑定顺序依次命名为 `s0`、`s1`、`s2`。如果你解绑中间的一个槽, 它后面的槽会自动重新编号向前补位——因为这个编号就是表达式里用的 `s` 变量本身。常见的坑: 你在 Scan 页或某个表达式里写死了 `s2`, 结果解绑了前面的槽, 原来的 `s2` 变成了 `s1`, 含义就跟着变了。实用提示: 尽量先把所有要扫的字段绑定齐再去写扫描表达式; 若必须中途解绑, 回头核对一遍各表达式里的 `s` 编号是否还指向你想要的字段, 并瞄一眼顶部 Scan 汇总确认槽数正确。

一句话记住扫描圆点

灰色空心环 = 普通数值; 点一下变橙色实心带编号 = 这个字段交给扫描槽 `s$n` 自动扫; 再点一下还原, 且记得你之前填的扫描值。编号即 `s` 变量, 解绑中间槽会让后面的槽整体前移。

4.3 Edit 页—period 卡片与 DAC 行（逐控件详解）

本节面向**从未见过本软件界面**的新用户，逐个讲清楚 Edit 页里一张「period 卡片」上的每一个控件。读完你应该能独立看懂、填写、并安全地编辑一段脉冲时序，而不需要旁人指导。

4.3.1 先理解：period 卡片是「时间轴」，不是表格

在 Edit 页里，每一个 period（时间段）都被画成一张**白色卡片**。这些卡片在一条**可以横向滚动的时间轴**上从左到右一字排开——请把它们想象成一条被切成若干段的时间带，而**不是**一张可以随意填格子的二维表格。

作用 一张卡片代表「一段连续的时间」。在这段时间里，每条数字通道要么一直是高电平、要么一直是低电平；每条模拟总线（DAC）保持某个码值或做某种变化。整条实验时序就是这些时间段一段接一段拼起来的。

卡片之间的顺序 卡片从左到右的排列顺序**就是**实际执行的先后顺序。最左边的卡片先发生，往右依次执行。

点击或拖动后会发生什么 **拖动卡片可以重新排序 period**。把一张卡片拖到另一张前面或后面，时序里这段时间的发生位置就跟着改变。这是调整实验流程顺序最直接的方式。

常见坑 因为是时间轴而不是网格，不要指望「上下对齐的格子」有某种行列关系——真正有意义的只有**左右顺序**。如果时间轴很长，记得用横向滚动条往右翻，后面可能还有卡片没显示出来。

实用提示 在动手填数值之前，先从左到右快速浏览一遍所有卡片，建立「这段实验先做什么、再做什么」的整体印象，再去抠每个控件的细节，会高效很多。

4.3.2 位置标签（卡片顶部）

每张卡片最上方有一行**位置标签**，例如 Period 2 / 5。

作用 它告诉你「这是第几张卡片，一共有几张」。Period 2 / 5 表示当前这张是第 2 个 period，整条时序总共有 5 个 period。

接受什么值 这是一个**只读的显示标签**，不是输入框，你不能点进去改它。它的数字会随着你增删、拖动 period 自动更新。

点击后会发生什么 它本身不响应编辑。它的价值在于「定位」——当你在一长串卡片里来回滚动时，靠这个标签快速确认自己正在看第几段。

常见坑 如果你拖动卡片重新排序，标签里的编号会立刻重排（原来的「Period 2」可能变成「Period 3」）。所以不要用编号去记忆某段的内容，内容才是它的身份。

实用提示 当总数（斜杠后的数字）和你预期的 period 数量不一致时，说明你可能多加或漏加了一段，借这个标签能第一时间发现。

4.3.3 Duration 输入框（带嵌入式扫描圆点）

紧接位置标签下面是 **Duration** 字段：一个文本输入框，右侧嵌着一个小小的**扫描圆点（SCAN DOT）**，就像数字微调框右边那个按钮一样。

作用 它决定「这一段时间持续多久」，也就是这个 period 的长度。

Period 2/5

Duration

100000



ns



name...

- ☒ trap
- ☐ cooling
- ☐ probe
- ☐ repump
- ☐ pushout
- ☐ grey_cooling
- ☐ emCCD

接受什么值 填一个**数字** (配合右边的单位下拉决定它是 ns/us/ms/s)。这个框**只接受数字、小数点、e 记数法与正负号**，其它字符在键入时直接被拒绝 (避免误输)。要让这一段时长**随扫描变化,不是手动输入表达式**——而是点右侧的扫描圆点把它绑定成扫描槽 (见下)。

嵌入式扫描圆点 输入框右侧那个圆点用来把这个 Duration 字段**绑定到一个扫描槽**。绑定后一切自动发生：该字段变橙、显示成 `s0/s1` 之类的符号，单位框自动切到内部的 `str (ns)` 并禁用——你不用、也无法手动选这个单位。表示「这段时长不是固定的，会在扫描的各个点上取不同的值」。再次点击圆点解除绑定、恢复成普通数值 (单位回到 ns/us/ms/s)。

点击或输入后会发生什么 填入一个数字并配好单位后，这段时间的实际长度立即按该数值生效；点了扫描圆点之后，这一段就变成可扫描的，实际时长在运行扫描时由对应扫描槽逐点提供。

常见坑 时长为 0 是不允许的——一个 period 必须有正的持续时间，填 0 会被拒绝。想让时长可扫描，**点扫描圆点**即可，无需去找“输入表达式”的入口、也无需手动切单位。

实用提示 改完 Duration 顺手看一眼单位：普通时长用 ns/us；橙色 `str (ns)` 是绑定扫描后自动出现的，不用手动碰。

4.3.4 单位下拉 (Duration 旁边)

Duration 输入框旁边有一个**单位下拉框 (combo)**。

作用 它决定 Duration 框里那个数值按什么时间单位解释。

接受什么选项 下拉里**只有 ns / us / ms / s** 四个数字单位。还有一个内部的 `str (ns)` (符号化/可扫描时长用)——但它**不在下拉里、不可手动选**：只有当你用扫描圆点把这段时长绑定成扫描槽时，单位框才会**自动**切到 `str (ns)` 并禁用。

点击后会发生什么 切换单位会改变 Duration 框里数字的量纲。例如同样填 `100`，选 `ns` 是 100 纳秒，选 `us` 就是 100 微秒，相差一千倍。

常见坑 最容易出错的是**本想填微秒却把单位留在了 ns** (导致时间短了一千倍)。每次改完 Duration，习惯性扫一眼单位框。

实用提示 要把这一段拿去扫描，直接点 Duration 框右侧的扫描圆点即可——单位会自动变成 `str (ns)`，无需任何手动单位操作。

4.3.5 通道复选框 (每条可见通道一个)

卡片中段是一排**复选框**，每一条当前可见的数字通道对应一个复选框。

作用 它控制「在这一段时间里，这条通道是高还是低」。**勾选 = 该通道在本 period 内为高电平 (HIGH)**；**不勾 = 低电平 (low)**。

接受什么值 只有「勾 / 不勾」两种状态，没有中间态。

点击后会发生什么 勾上某通道，本段时间里这条线就被拉高；取消勾选则保持低。整段时间内电平不变——要让一条通道在某时刻翻转，就靠相邻 period 之间该复选框勾选状态的不同来实现。

标签被省略的情况 通道标签如果太长，会被用省略号 ... 截断显示；**完整名字在鼠标悬停的 tooltip 里**。所以看不清全名时，把鼠标停在标签上稍等即可看到完整通道名。

常见坑 因为标签会被省略，不同通道的截断结果可能看起来很像，容易勾错行。下手前最好靠 tooltip 确认这一行到底是哪条通道。

实用提示 把「这条通道在哪几段为高」想成在时间轴上画一条方波：相邻卡片勾选状态相同就是电平保持，状态变化的地方就是一个上升沿或下降沿。顺着卡片从左到右看这一列复选框，就能一眼读出该通道的整个波形。

4.3.6 DAC（模拟总线）行—mode 下拉

卡片下部，**每一条模拟总线（DAC）** 各占一行，例如 `da_dipole` 或线圈的 `da_x/da_y/da_z`。每行由两个控件组成：一个 mode 下拉 + 一个 value 输入框。先看 mode 下拉。

总线行从哪来：设备自带的通道标签

一条模拟总线是由若干**位通道**拼出来的（例如三路线圈各占一段 6 bit）。编辑器把这些位通道**折叠**成一条总线行，依据的是**设备自带的通道标签**——真机的标签来自板卡 XDC（如 `da_dipole[0]`），虚拟序列器同样自带一份对齐的标签。所以**从会话打开的编辑器（`exp.pulse\_gui()`）一开始、还没载入任何模板时**就已经把线圈显示成 `da_x/da_y/da_z` 三条总线行，和真机同构——不是要先载一个模板才折叠。标签只标注设备**真正携带**的那些通道，绝不广告一条成员缺席的空总线。

作用 mode 下拉决定「这一段时间里，这条 DAC 的值如何变化」。

接受什么选项 三个选项：`Edge` / `Ramp` / `Hold`。

各选项的含义 **Edge** 在本 period **开始的瞬间直接跳到** value 框里写的那个值，然后整段保持该值。适合需要在段首做阶跃变化的场合。

Ramp 在本段时间内**从上一段的值线性渐变到**本段 value 框里的目标值。适合做平滑扫频/缓变控制。

Hold **保持上一段的值不变**（本段不做任何改变），此时这一行的 value 等于沿用之前的码值。

点击后会发生什么 选 **Edge** 会让 value 框里的数立刻在段首生效；选 **Ramp** 会让输出在这一整段里从「前值」滑到「本段值」；选 **Hold** 则忽略本段 value、延续旧值。

常见坑 **Ramp** 的起点是「上一段结束时的值」，不是 0，也不是本段框里的数——如果你忘了前一段的值是多少，渐变的起点可能和预期不同。**Hold** 模式下你在 value 框里改数往往不起作用，因为它本就保持旧值。

实用提示 想要一个干净的阶跃，用 **Edge**；想要一条斜坡，把斜坡终点写进本段 value 并选 **Ramp**，同时确认前一段已经把起点值设好；只想「这段别动这条 DAC」就选 **Hold**。

4.3.7 DAC（模拟总线）行—value 输入框（带嵌入式扫描圆点）

每条 DAC 行的第二个控件是 **value 输入框**，同样在右侧嵌了一个**扫描圆点**。

作用 它存放这条 DAC 在本段要输出的**原始 10 位 DAC 码**。

接受什么值 一个 **-512 到 +511 之间的带符号整数**（10 位总线）。**0 = 真零点（驱动器输出 0 V）**，负数输出负电压、正数输出正电压。这个框**只接受整数**（可带负号；小数点、字母在键入时被拒绝），并把值约束在 **[-512, +511]**。线上实际传输的是 offset-binary 码（= 值 + 512，0 V 即码 512），由编译器自动换算——你只管填带符号 LSB 数。

嵌入式扫描圆点 和 Duration 一样，右侧圆点用来把这个 value**绑定到扫描槽**。一旦绑定，框里会显示成 `s0/s1` 之类的符号并**变成橙色**，表示这条 DAC 的码值会在扫描的不同点上取不同值。再点一次圆点解除绑定、恢复成普通数字。

点击或输入后会发生什么 填入 $-512..+511$ 的带符号整数后，配合本行 mode (**Edge** 段首跳到该码 / **Ramp** 渐变到该码) 这个码值就会生效；点了扫描圆点绑定到扫描槽后，运行扫描时由对应槽逐点提供码值。

常见坑 值**超出 $-512..+511$** 是无意义的——10 位 DAC 只有 1024 个码 (**偶数个**，不存在“正中间的码”；约定 0 V = 线上码 512，所以负向比正向多 1 LSB)。别把它当电压填 (要填的是对应电压的带符号 LSB 数)。在 **Hold** 模式下，这个框显示**沿用进来的值** (只读)。

实用提示 **+511** 是正满量程、**-512** 是负满量程、**0** 是真零点 (0 V) ——这就是刻度本身。未驱动的总线 (以及运行间隙) 自动停在 0 V ，preview 里 0 V 参考虚线位于模拟行的正中，负值画在虚线下方。要扫描一条 DAC，直接点它 value 框右边的圆点绑定到扫描槽即可，绑定后橙色高亮会让你一眼看出「这条值是被扫描的」。

一张卡片就是一段时间的全部状态

把一张 period 卡片读完，你就知道了这段时间的「全部信息」：它持续多久 (Duration + 单位)、每条数字通道是高还是低 (复选框)、每条模拟总线输出什么码、怎么变 (DAC 行的 mode + value)。整条实验时序，无非是把这样一张张卡片按时间轴从左到右拼接起来。**时长不能为 0、DAC 填的是 $-512..+511$ 的带符号 LSB ($0 = 0\text{ V}$) 而非电压、拖动卡片即可重排顺序**——记住这三条，基本就不会在 Edit 页踩坑。

4.4 Edit 页—底部 Control / Connection / Channels 控制条 (逐按钮详解)

Edit 页 (脉冲编辑页) 的最下方有一整条横向工具区，分成三张带标题的卡片：左侧是 **Control** (运行与文件控制)，中间是 **Connection** (选连什么时序器后端)，右侧是 **Channels** (通道显示控制)。如果你以前从没用过这个界面，可以把它想象成「磁带录音机的操作面板」：上半部分是你画脉冲波形的大画布，而这一条底部工具区就是控制「录、放、停、存、读」、「接哪台机器」以及「让画面里显示哪些轨道」的按钮。下面对每一个按钮、每一个下拉框逐一讲清楚：它是干什么的、能接受什么值、点下去之后会发生什么、容易踩的坑，以及一个实用小提示。

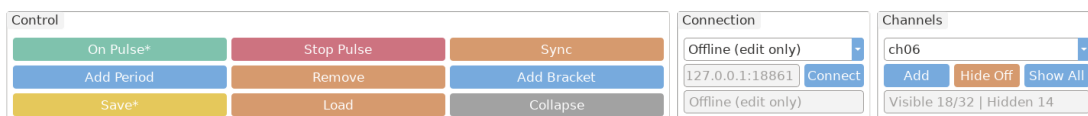


Figure 4.5: Edit 页底部的三张控制卡片——左为 Control (9 个按钮, 3 行 3 列), 中为 Connection (后端下拉 + host:port + Connect + 状态行), 右为 Channels (下拉框 + 三个按钮 + 一个只读统计标签)。

4.4.1 Connection 卡片——打开后再选连什么

打开 GUI 后 在这里挑时序器后端，不必在命令行里事先定死。下拉三选一：**Virtual (sim)** (本地内存时序器，可离线编辑 + 模拟 On Pulse)、**Remote server** (连一台正在跑的 FPGA 时序器服务，填 host:port)、**Offline (edit only)** (只编辑、不调后端)。选 Remote 时下方 host:port 输入框才可编辑；点 **Connect** 应用所选连接，状态行显示当前连到哪。换后端只换 On Pulse / Stop / Sync 背后说话的对象，**用编辑器自己的通道**重建时序器——通道布局不会被打乱；连不上会弹出提示并保持原连接不变。默认启动即 Virtual (命令行 pulse_gui.py 无参数时)，显式 **-remote-host** 才在启动时直连。

4.4.2 Control 卡片：运行控制按钮 (Stop / On)

Control 卡片一共有 9 个按钮，排成 3 行 3 列。最重要、也最需要先认识的是这两个带颜色的运行按钮。

Stop Pulse (红色) 作用：把测序器 (sequencer) 切换到**安全空闲状态**，也就是让所有硬件输出全部归零 (所有通道变为低电平 0，DAC 总线也回到安全值)。**接受什么**：这是一个纯动作按钮，没有任何输入框，直接点一下即可，没有参数可填。**点下去会发生什么**：当前正在跑的脉冲序列会被立即叫停，FPGA 把全部输出拉到 0，进入待命。**常见坑**：很多新手以为「关掉 GUI 窗口」就等于停止输出，其实不是——窗口关掉之后硬件可能仍按上一次的指令在跑。要真正让设备安静下来，必须显式点 Stop Pulse。**实用提示**：把它当成「急停按钮」。只要你发现实验行为不对、激光或线圈有异常、或者准备离开设备，第一反应就是点这颗红色的 Stop Pulse，让一切归零，再慢慢排查。

On Pulse (绿色) 作用：这是「开始运行」按钮，而且它一颗按钮做两件事——先 **prepare** (把当前编辑好的脉冲表上传/同步到测序器)，**然后立刻 start** (启动它开始输出)。**接受什么**：同样是纯动作按钮，无输入参数。**点下去会发生什么**：你在画布上编辑的所有周期 (period)、通道延时、扫描设置等会被编译并写进 FPGA，写完后序列马上开始跑。**星号含义**：按钮文字会在 **On Pulse** 与 **On Pulse*** 之间切换 (confocal 同款语义)——**带星号 = 点下去会应用新的东西**：编辑器里有设备上没有的修改 (UNSYNCED，状态点同时变橙)、设备已停止/出错、或还从未运行过；**不带星号**只有一种情况：设备正在运行且跑的就是你眼前的参数。按钮颜色**始终是绿色**，不会用颜色表达状态 (避免和底部其他常驻橙色按钮混淆)。**常见坑**：「上传」与「启动」不是两步——On Pulse 已合并两者，每次点击都用**最新**内容重新上传并运行，旧内容不会残留 (需要反向同步时用 Sync 按钮把设备上的脉冲拉回编辑器)。**实用提示**：看到 **On Pulse*** 就表示「眼前 $\neq$ 设备」，点一下即生效；运行中改了参数又改回原样，星号会自动消失。

红绿配色的含义

红色 = 让一切停下并归零 (安全)，绿色 = 上传并开始运行。养成「先看颜色再点」的习惯，可以避免在不该启动的时候误触发输出。

4.4.3 Control 卡片：周期编辑按钮 (Remove / Add Period / Add Bracket)

这一组按钮用来增删脉冲表里的「周期 (period)」结构，也就是改变时间线的骨架。

Remove 作用：删除**当前选中的 (或最后一个)** 周期。**接受什么**：动作按钮，无输入。**点下去会发生什么**：被选中的那个周期会从时间线上消失，后面的周期相应前移。**常见坑**：如果你没有特意选中某个周期，它删除的是「最后一个」周期，所以可能删掉的并不是你以为的那一段；删之前先确认你选中的是哪一个。**实用提示**：删错了不要慌——只要还没保存，可以再用 Add Period 补回一个空周期，再重新填内容；养成删除前先 Save 一次的习惯能让你随时回退。

Add Period 作用：在脉冲表末尾**追加一个新的周期**。**接受什么**：动作按钮，无输入。**点下去会发生什么**：时间线右侧会多出一个新的 (通常是空白/默认) 周期卡片，你可以继续在里面设置各通道的开关与时长。**常见坑**：新周期是加在**末尾**的，不是插在你当前选中的位置中间；如果你需要在中间插入，得调整顺序或重新规划。**实用提示**：搭建一个新序列时，先用 Add Period 把需要的周期数量一次性建够，再逐个填内容，比边建边填更不容易乱。

Add Bracket 作用：**切换 (toggle)** 一个重复括号——也就是在一段连续的周期范围上加一个**有限次的内层循环** (重复 $\times N$ 次)。**接受什么**：动作按钮 (toggle 性质，再点一次可取消)。**点下去会发生什么**：被括起来的那几个周期会作为一个内循环重复 N 次，而整张脉冲表本身**仍然可以无限重复** (外层 $\times \infty$)。换句话说，你能得到「整体永远循环、其中某几段又内部重复 N 次」的嵌套结构。**常见坑**：要分清**内层有限循环**和**外层无限循环**是两回事——括号只控制内层那几段的重复次数，不影响整表是否一直跑下去。另外要注意，括号定义的是一段**连续范围**，跨度选错会把不想重复的周期也圈进去。**实用提示**：需要「先重复一组装载/冷却脉冲若干次，再继续后续步骤」时，Add Bracket 就是你的工具；不需要时再点一次把它取消即可，因为它是可切换的。

4.4.4 Control 卡片: Collapse (折叠左侧面板)

Collapse 作用: 把左边的 **Channel Names (通道名)** 面板和 **Delay / Scan (延时/扫描)** 面板**隐藏起来**, 从而把腾出来的横向空间全部让给中间的周期时间线, 让波形画布更宽、更好看。**接受什么:** 动作按钮 (切换性质), 无输入。**点下去会发生什么:** 左侧两块面板收起, 时间线立刻变宽; 同时这颗按钮的文字会从「Collapse」变成「**Show Left**」, 并且界面上会留下一个小小的存根 (stub), 让你随时能把面板再调回来。**常见坑:** 折叠之后通道名和延时编辑区暂时看不见了, 新手容易以为「设置丢了」——其实只是被藏起来, 数据都还在, 点 Show Left (或那个小存根) 就回来了。**实用提示:** 当你的序列周期很多、时间线被挤得很窄看不清细节时, 先 Collapse 腾地方专心看波形; 需要改通道名或延时时再 Show Left 调回来。

4.4.5 Control 卡片: 文件操作 (Save / Load)

Save 作用: 把当前脉冲保存成**完整的一包**磁盘文件。**接受什么:** 动作按钮, 无需在按钮上输入参数 (实际文件名通过保存流程确定)。**点下去会发生什么:** 它会写出 **.json** 脉冲本体 + **.png** 预览图 + **<name>_program.json** (编译好的、FPGA 可直接上载的程序——**无论是否扫描都会保存, 与 On Pulse 实际上传的内容一致**); 如果当前存在扫描表 (scan table), 还会再写出 **<name>_scan.npy** 数组文件。也就是说一次 Save 产出三到四个配套文件, 方便你完整复现这次实验设置; 某一件写失败时状态栏会写明跳过了哪件及原因。**状态指示 (confocal 同款):** 有**未保存的修改**时按钮显示**黄色**且文字带星号 (**Save***), 标题栏同时带星号; 内容保存干净后变回**蓝色的 Save**。**实用提示:** 把「**Save 蓝色 + 标题栏星号消失**」当作「已存盘」的可靠信号; 运行前如果看到 **Save*** 还是黄色, 说明你最新的改动还没落盘, 建议先 Save 再 On Pulse, 避免事后找不到这次用的是哪份配置。

Load 作用: 从磁盘**载入**一个脉冲 **.json** 文件, 并把**保存时的整包关联一起恢复**。**接受什么:** 动作按钮; 会让你选择一个之前保存过的 **.json**。**点下去会发生什么:** 选中的脉冲文件被读入, 画布上的周期、通道、延时、扫描等设置会替换成文件里的内容; 若同目录有配套的 **<name>_scan.npy**, 它会被自动重新挂回“载入文件”来源 (Delay / Scan 面板显示该文件名)。**误选了 _program.json 或 _scan.npy 时**, 只要同目录有对应的脉冲 **.json**, 会自动改为加载脉冲本体并提示。**常见坑:** Load 会用文件内容**覆盖**当前编辑区——如果你当前有未保存的改动 (Save 还是黄色、标题栏有星号), 载入前务必先 Save, 否则改动会丢失。**实用提示:** Save 与 Load 是一对——一包文件同名成套 (**.json、.png、_program.json、_scan.npy**), 日后一眼认出哪份配置对应哪次实验, 复现和归档都更省心。

一次 Save 到底产生几个文件?

通常三个: **.json** 脉冲本体 + **.png** 预览图 + **<name>_program.json** 编译好的 FPGA 程序 (**无论是否扫描都有**)。设置了扫描表时是四个 (再加 **<name>_scan.npy** 扫描数组)。看到 Save 变蓝、标题栏星号消失, 就代表这一套都已经写好了。

4.4.6 Channels 卡片: 把隐藏通道调回来 (下拉框 + Add)

右侧的 Channels 卡片专门管「画面里显示哪些通道」, 它不改变硬件输出, 只改变**你看到什么**。

隐藏通道 / DAC 总线下拉框 作用: 这是一个下拉列表, 里面列出当前**被隐藏的**通道以及 DAC 总线, 供你挑一个重新显示出来。**接受什么:** 从下拉项中选择一个条目 (某个被藏起来的通道或某条总线)。**点下去会发生什么:** 仅仅是「选中」, 此时还没真正显示——它只是告诉旁边的 Add 按钮「我要调回的是这一个」。**常见坑:** 光在下拉框里选好并不会让通道出现, 必须再点 Add 才生效; 如果下拉框是空的, 说明当前没有被隐藏的通道可调回。**实用提示:** 先在下拉框里看一眼到底有哪些通道被藏了, 这

本身就是个快速排查「我的某个通道怎么不见了」的好办法。

Add 作用：把下拉框里**当前选中**的那个通道/总线**重新显示**出来。**接受什么：**动作按钮，配合上面的下拉框使用。**点下去会发生什么：**被选中的通道立刻回到时间线画布上，重新可见可编辑。**常见坑：**注意这是「显示/隐藏」层面的操作，把通道调回来**不会**改变它的脉冲内容，也不会改变硬件输出；别把它误解成「新增一个硬件通道」。**实用提示：**配合下面的 Hide Off 一起用——先 Hide Off 把无关通道清掉，等需要某个特定通道时再用「下拉框选中 + Add」精准把它单独调回来。

4.4.7 Channels 卡片：批量显示/隐藏 (Hide Off / Show All) 与统计标签

Hide Off 作用：隐藏所有「全程为关」的通道——也就是那些在**每一个周期里都处于关（低电平 0）**的通道，用来减少画面杂乱。**接受什么：**动作按钮，无输入。**点下去会发生什么：**凡是从头到尾都没被用到（始终是 0）的通道会从画布上消失，剩下的都是真正参与了这个序列的通道，画面立刻清爽很多。**常见坑：**判断标准是「在**每个**周期都为关」——只要某个通道在**任意一个**周期里有过高电平，它**不会被**隐藏。所以如果你期望某条线被藏却没被藏，多半是它在某个周期里其实有输出。**实用提示：**序列搭好后先点一下 Hide Off，能把几十条通道里真正在用的那几条凸显出来，审阅波形时效率高很多；想看全貌时再用下面的 Show All 还原。

Show All 作用：显示所有硬件通道，把之前隐藏的全部调回来。**接受什么：**动作按钮，无输入。**点下去会发生什么：**所有硬件通道（无论之前是被 Hide Off 还是被单独隐藏的）一次性全部重新出现在画布上。**常见坑：**通道一多画面会变得很密集、需要滚动，这是正常的；Show All 是「看全貌」用的，不适合长时间编辑细节。**实用提示：**把 Hide Off 和 Show All 当成一对开关——**Hide Off 聚焦、Show All 总览**。检查「是不是漏配了某个通道」时点 Show All，专注编辑时点 Hide Off。

只读统计标签 作用：这是一个**不可编辑**的文字标签，实时显示当前的可见/隐藏统计，例如「Visible 18/32 | Hidden 14」。**接受什么：**它不接受任何输入，纯展示。**点下去会发生什么：**什么都不会发生——它只是一面「仪表」，告诉你 32 条硬件通道里当前显示了 18 条、藏了 14 条。**常见坑：**别试图点它或改它，它不是按钮也不是输入框。**实用提示：**用它快速核对——比如你 Hide Off 之后看到「Visible 6/32」，就能立刻知道这个序列实际只用到了 6 条通道；数字和你的预期不符时，说明有通道被多配或漏配，值得回头检查。

Channels 卡片只影响「显示」，不影响「输出」

请牢记：Channels 卡片里的下拉框、Add、Hide Off、Show All 全部只改变**画面上看到哪些通道**，丝毫不改变 FPGA 实际输出的脉冲。把一条通道藏起来，它在硬件上该高还是高、该低还是低；调出来也只是让你重新看见而已。真正改变运行行为的，是左边 Control 卡片里的 On Pulse / Stop Pulse。

4.5 Preview 页与扫描高亮在预览里怎么显示（逐元素详解）

Preview（预览）页是你在真正把脉冲发到 FPGA 之前，用来“用眼睛检查一遍”序列长什么样的地方。它把你 Edit 页里搭好的周期表（period table）画成一张时间图：横轴是时间，纵轴是一行一行的通道。本节假设你**从来没见过这个界面**，所以我们会把页面上每一个开关、按钮、每一种颜色的含义都讲透——它们各自**有什么作用、接受什么值、点下去或切换后会发生什么、常见的坑**，以及**一个实用提示**。

4.5.1 它什么时候重画：没有“刷新”按钮

在你动手找按钮之前，先记住一件最重要的事：**Preview 页是自动重画的，根本没有手动刷新按钮。**

自动重画的时机 只要你**切换到 Preview 这个标签页**，或者**在本页里拨动任何一个开关**，整张图就会立刻重新生成一遍。你不需要、也找不到一个“刷新”“重画”“更新预览”之类的按钮——因为根本不存在。

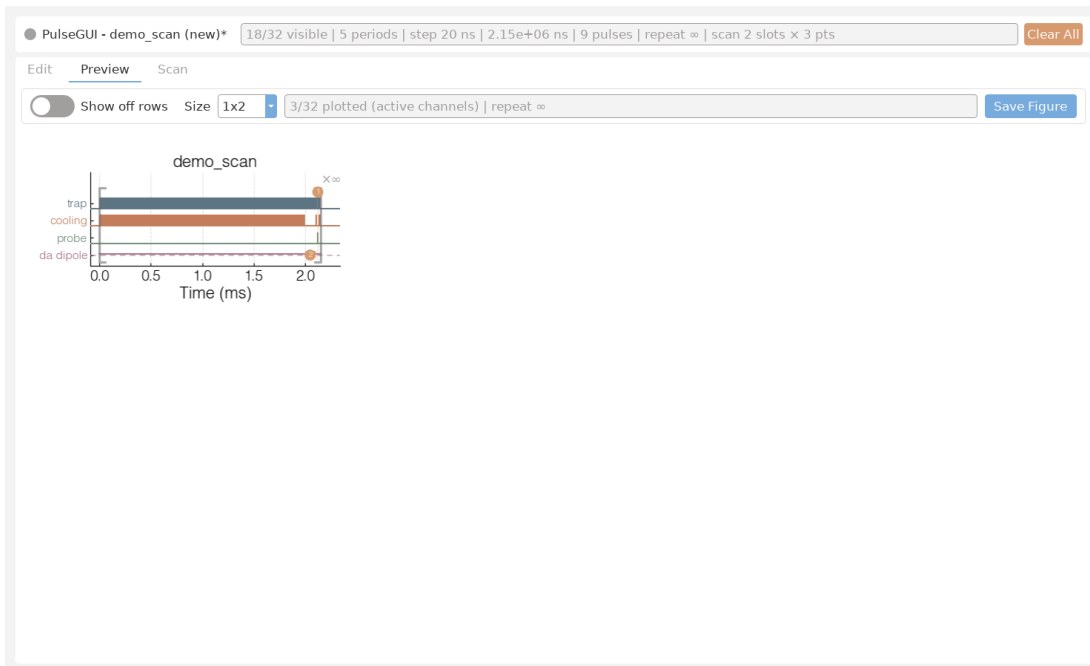


Figure 4.6: Preview 页整体外观：顶部是“Show off rows”开关、状态标签与“Save Figure”按钮，下方是脉冲时间图；图中橘色透明区域就是扫描高亮，每个高亮带着它的 1 基槽位编号。

画的是哪一份数据 它画的是**未展开的周期表**，也就是**只画一次循环（one iteration）**的样子。如果你的序列里有“重复 N 次”这种结构，预览**不会**把它真的铺开成 N 份重复的波形，而是画一次、再用括号和倍数标注（详见后文的 repeat 小节）。这样图不会因为重复几百次而变得无法阅读。

常见的坑 新手最容易困惑的是：“我在 Edit 页改了参数，回到 Preview 怎么还是旧的？”——只要你是**切换标签页**过来的，它就已经按最新数据重画了。如果你怀疑没更新，最稳妥的办法是切走再切回 Preview，强制触发一次重画。

实用提示 边调参数边看效果时，养成“改完就切回 Preview 瞄一眼”的习惯。因为重画是免费且自动的，多看几次不会有任何代价，反而能在发脉冲前就抓出明显的错误（比如某个通道一直是空的、某个延迟把边沿推到了奇怪的位置）。

4.5.2 “Show off rows” 开关：要不要画一直关着的通道

这是页面顶部的一个开关（toggle），标签写着 Show off rows（显示关闭的行）。

作用 它决定预览图里**要不要把那些自始至终都没亮过的通道也画出来**。

接受什么值 它是一个二选一的开关，只有两种状态：**关 (off)** 和**开 (on)**。没有中间档，也不接受数字输入。

关 (off) 时会发生什么 这是默认、也是最常用的状态。图里**只画那些至少高电平过一次**的通道——也就是“在这次循环里干过活”的行。一直保持低电平、从头到尾没亮过的通道会被**隐藏**，图会显得干净、紧凑，你能把注意力放在真正有脉冲的通道上。

开 (on) 时会发生什么 一旦打开，图里**连那些一直关着的通道也会画出来**。结果就是多出一大堆**空荡荡的行**（这些行从头到尾没有任何填充的条），图会一下子变高、变长。它的用处是：当你想确认“某个我以为该用上的通道，是不是其实一直没被触发”时，打开它就能一眼看到那条空行确实在、且确实是空的。

常见的坑 打开后图里突然冒出几十行空白，新手会以为“出错了”或“通道全废了”。其实那只是把平时藏起来的常关通道显示出来而已，属于正常现象。看完记得拨回去，否则后续每次预览都拖着一堆空行，不好读。

实用提示 日常调试让它**保持关**，图最清爽；只有在排查“为什么某通道没动静”时才临时打开。开 → 关来回切换是安全的，图会稳定地回到一致的状态，不会因为切换次数多了就乱掉。

4.5.3 状态标签与“Save Figure”按钮

在开关旁边还有两样东西：一个**状态标签**和一个 **Save Figure**（保存图像）按钮。

状态标签（作用） 这是一行文字提示，用来告诉你当前预览的状态信息。它不接受输入、也不能点——纯粹是给你看的。当你不确定图是不是最新、或者保存动作有没有成功时，瞄一眼这里通常能得到线索。

Save Figure 按钮（作用） 点它可以把**当前这张预览图原样保存成一个.png 图片文件**。这样你就能把波形截图贴进实验记录、汇报或手册里，而不必自己去截屏。

点下去会发生什么 按下后，程序会把现在屏幕上看到的这张图导出为一个 **.png** 文件。导出的内容就是当下的样子——包括你此刻的“Show off rows”状态和所有橘色扫描高亮，所见即所存。

常见的坑 因为存的是“当下这一刻”的图，所以**保存前要把图调成你想要的样子**：该开/关的行先设好，确认扫描高亮显示正确，再点保存。如果你保存完才发现某通道没显示出来，多半是“Show off rows”的状态不对，调好再存一次即可。

实用提示 给保存的文件起个能认出实验配置的名字（比如带上序列名或日期），方便日后翻找。配合本手册前面提到的“保存脉冲会一起产出 pulse.json + png + scan 数组”的流程，这里手动导出的 png 适合用来单独抓某一帧你特别想留档的预览。

4.5.4 图本身：数字通道行与模拟（DAC）行怎么画

图的主体是一格一格的行。理解这张图的关键是分清**两类行**：数字通道行，和模拟（DAC 总线）行。

数字通道行（每个通道一行） 每一个数字通道占**一行**。在这条行上，凡是该通道处于**高电平**的时间段，都会画成一条**填充的实心条（bar）**；低电平的时间段则是空白。所以你横着看一行，就能读出这个通道在一次循环里“什么时候开、什么时候关、开多久”。

模拟（DAC）总线行（一条总线一行） 每一条模拟 DAC 总线只占**一行**，而且画法和数字行不同：它画的是一条**阶梯线（step line）**，这条线的高度**跟随该总线的真实数值**上下走。数值为 0 时，线**贴在基线上**；数值越大，线抬得越高。请特别注意：**每条模拟行只有一条线，不是两条**——不要去找“上下两条边界线”，就是一条随值起伏的台阶线。

总线的比特通道去哪了 一条 DAC 总线在底层是由很多比特（bit）通道拼出来的。在预览里，这些**比特通道总是被折叠进那一条模拟行，绝不会被单独拆成一行一行的数字通道显示**。换句话说，你在数字行区域**看不到** DAC 的某个 bit；它们全部归并到那条阶梯线里。

常见的坑 新手常犯两个错：一是想在数字行里找 DAC 的某根 bit，结果找不到——那是设计如此，bit 永远折叠进模拟行；二是看到阶梯线贴着底就以为“这行没在工作”，其实贴底只代表当前值是 0，线还是在的。

实用提示 读模拟行时，把注意力放在**台阶的高度变化**上：每一级台阶就是一次新的 DAC 取值。想确认某段时间 DAC 是不是输出了你设定的码值，顺着那条线找对应时间点的高度即可。

4.5.5 repeat (重复): $\times\infty$ 、 $\times N$ 与那个大括号

预览只画一次循环，但它会用**标注**告诉你这段序列实际会重复多少次。

无限循环: $\times\infty$ **加一个大括号** 如果整段序列是**无限重复**的，图上会显示 $\times\infty$ ，并配一个**横跨整段序列的大括号 (bracket)**，把这段会被反复执行的区域整个圈起来。这个括号是在告诉你：“括号里的内容会一遍又一遍地跑下去。”

有限内层循环: $\times N$ 如果某一段是**有限次**的内层循环，它会标成 $\times N$ (N 是具体的重复次数)，同样用括号圈出它覆盖的范围。于是你能同时看到“外层无限 $\times\infty$ ”和“内层若干次 $\times N$ ”两层标记并存。

永远不会出现“inf”这个词 请注意，图里**从不出现字面的英文“inf”**。无限重复**只用** $\times\infty$ 这个符号表示。如果你在哪儿看到了“inf”，那不是这套预览该有的输出。

延迟把边沿推到表尾之外时会怎样 这是一个容易吓到新手的细节：通道延迟 (delay) 可能把某个边沿**推到周期表末尾之后**。这种情况下， $\times\infty$ 的大括号会自动延伸，把这个被推出去的边沿也包进来，**绝不会有任何东西画在循环括号之外**。直观地说就是“括号会自己变长去兜住那条迟到的边沿”，让整张图读起来始终是物理上自治的——一切都在循环之内。

常见的坑 如果你没意识到延迟会延长括号，可能会困惑“为什么这个 $\times\infty$ 括号比序列正文还长一点”。那正是延迟造成的，是正确行为，不是 bug。

实用提示 看 repeat 标记时，先找最外层的 $\times\infty$ 确认整体是不是无限跑，再逐个看内层的 $\times N$ 核对每段重复次数对不对。如果发现括号右端伸得比你预期长，去检查相关通道的延迟设置，多半就是它把边沿推出去了。

4.5.6 扫描高亮 (橘色透明): 三种被扫描的量怎么标

这是 Preview 页最有特色的部分。当你把某个参数绑定为**扫描槽位 (scan slot)** 时，预览会用**透明的橘色**把这个参数在图上对应的位置高亮出来，并**在该高亮上标一次它的槽位编号**。编号是**1 基 (1-based)** 的——也就是从 1 开始数，而不是从 0；每个高亮只标**一次**编号，不会在同一段上重复盖好几个相同的数字。

下面分三种被扫描的量来讲，它们的高亮形状和编号位置各不相同，认清形状就能一眼分辨你扫的到底是哪一类量。

被扫描的 DURATION (周期时长) 当你扫描某个周期的**时长**时，高亮是一条**橘色的竖向带子**，盖住**这个周期所占的那段时间**。关键点：这条带子在纵向上**只覆盖通道行的区域**——它的**顶部恰好到最上面那条通道的顶**，**绝不会再往上越过去**盖到标题那一带。它的**槽位编号**写在**最顶上通道正上方那一小条空隙 (headroom)** 里，刚好夹在通道和标题之间，**不会和标题文字叠在一起**。这样你既能看清是哪个周期被扫，又不会让数字糊进标题。

被扫描的 DELAY (通道延迟) 当你扫描某个通道的**延迟**时，高亮是一条橘色带子，盖在**这个通道自己脉冲的“前导段 (lead-in)”**上，而且**就画在这条通道自己的行上**——它会**精确落到正确的那条通道行**，不会跑到别的通道去。**槽位编号**也写在这条通道的行上。所以当你看到某条通道行的脉冲前面有一段橘色，就知道这条通道的延迟正被扫描。

被扫描的 DAC 值 当你扫描某条 DAC 总线的**输出值**时，高亮**跟着那条模拟阶梯线走**：它是一段**位于该数值高度上的、短短的橘色线段**，只覆盖被扫描的那个周期，**而不是把整行涂成一个色块**。也就是说，橘色出现在“线当前的高度”上，随值高低而高低。它的**槽位编号**写在这条模拟行的正下方一点点的位置。

怎么靠形状区分三种扫描

快速心法：橘色**满高地盖住整段时间**、数字在**顶部通道上方** = 在扫周期时长；橘色**只盖某条通道脉冲的开头**、数字在**那条通道行上** = 在扫这条通道的延迟；橘色是**一小段贴着阶梯线高度的短线**、数字在**模拟行下方** = 在扫**DAC 值**。记住这三种形状，你不用点任何东西就能读出自己到底在扫什么。

编号为什么只出现一次

即使一个被扫描的量在一次循环里跨了好几段（比如某通道在多个周期里都亮），它的槽位编号也**只标在第一段上**，其余各段**只上橘色、不再重复写数字**。这是有意为之，目的是避免同一个编号在一条通道上盖出好几个重叠的数字、把图弄花。所以**一个槽位在图上恰好对应一个数字**——看到几个不同的数字，就代表你正在同时扫几个不同的槽位。

4.6 Scan 页与扫描 workflow (逐元素详解 + 实操配方)

Scan 页是你**搭建扫描表**（也就是扫描点列表）的地方。所谓“扫描表”，本质上是一张二维数表：每一**行**代表实验要采集的一个**扫描点**，每一**列**代表一个被绑定的扫描槽（slot，记作 **s0**、**s1**、**s2**）。当你在硬件上 On Pulse 时，序列器会从上到下逐行执行这张表，每一行对应硬件上的一次无缝扫描点，并且**每个扫描点恰好发一次相机触发**。换句话说，扫描表有多少行，这一轮就采集多少个数据点。

在动手之前，先记住贯穿整页的核心约束：扫描表里你能引用的变量只有 **s0** 到 **s{n-1}**，没有历史遗留的 **x/y** 概念。每一列对应哪个物理量，是你在 Edit 页给某个字段点“扫描圆点”绑定时决定的——绑定第一个字段得到 **s0**，第二个得到 **s1**，以此类推。Scan 页本身不创建槽，它只负责给这些已存在的槽**填数值**。

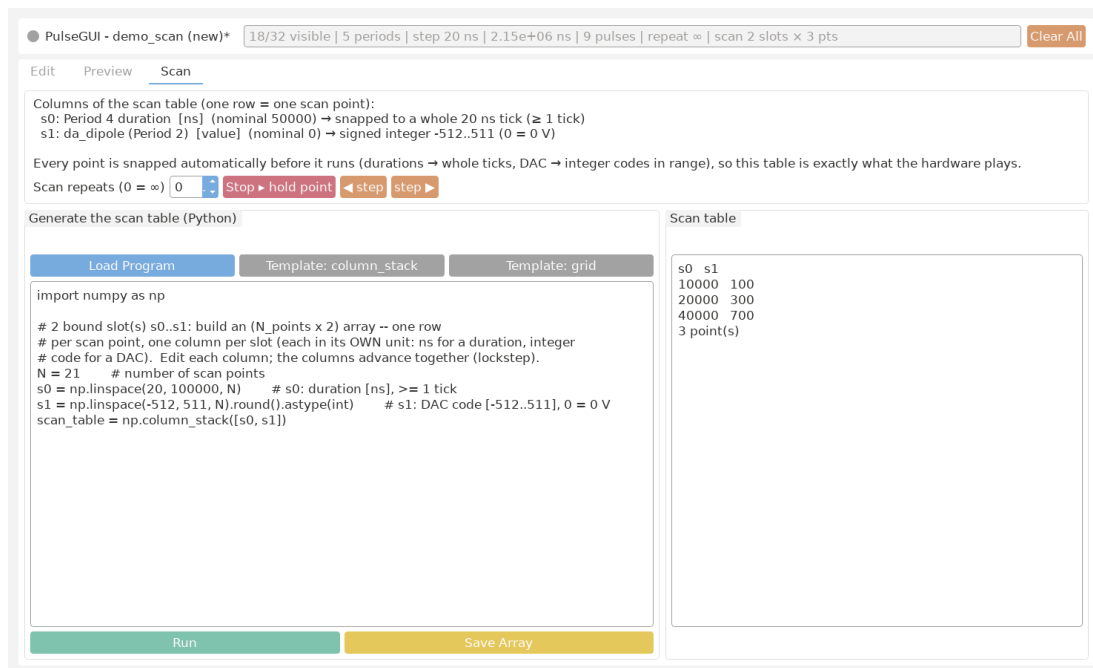


Figure 4.7: Scan 页整体布局：顶部是各列含义说明，中间是白色 Python 代码编辑器（其上方有 Load Program 与两个模板按钮），下方是 Run / Save Array 按钮和只读的扫描表预览。

4.6.1 顶部：扫描表列含义说明 (Columns of the scan table)

页面最顶部是一块**只读的说明文字**，每一个被绑定的扫描槽占一行，告诉你这张表的每一列到底代表什么物理量、属于哪个周期或通道、以及它的**单位**。

它显示什么 每行形如 `s0: Period 4 duration [ns] (nominal 50000)`、`s1: da\_dipole (period 2) [value]`。冒号前是槽号 (与代码里要用的变量名——对应)，冒号后是这个槽绑定到的字段的人类可读描述：是哪个周期的 duration，还是哪个周期的某个 DAC 总线值。

单位约定 方括号里的单位非常关键。时间类的槽 (duration、delay) 单位是 `[ns]` (纳秒)，你在代码里填的数字就按纳秒理解；DAC 类的槽单位是 `[value]`，表示**带符号 DAC 值** (LSB, 0 = 0 V)，不是电压、也不是纳秒，取值范围是整数 -512 到 +511 (线上自动换算成 offset-binary 码)。把这两类单位搞混是新手最常见的错误。

nominal 标注 对于像 duration 这样有标称值的槽，括号里会附带 (`nominal 50000`)，告诉你这个字段在 Edit 页填的原始值是多少。它是一个**参照基准**：你可以围绕这个标称值设计扫描范围 (比如以 50000 ns 为中心上下扫)，心里好有个数。

点击后会发生什么 这块区域是纯展示，**不能点击、不能编辑**。它会随着你在 Edit 页绑定/解绑字段而自动刷新行数和内容——你在 Edit 多绑一个字段，这里就多出一行 `s\{下一个号\}`。

常见坑 如果这里**一行都没有** (显示类似“无扫描槽”的占位提示)，说明你还没在 Edit 页绑定任何字段，此时写再多代码也没有列可填。请先回 Edit 页点亮至少一个扫描圆点。

实用提示 写代码前，先照着这块说明从上往下数一遍：第 0 行就是 `scan\_table` 的第 0 列，第 1 行就是第 1 列……让脑子里的“列序”和说明里的“槽序”对齐，能避免把 duration 的数填进 DAC 列这类张冠李戴的错误。

4.6.2 中间：Python 代码编辑器

中间一大块**白底等宽字体**的区域是 Python 代码编辑器。你在这里写一小段 Python 代码，**把一个形状为 `N_points × N_slots` 的数组赋值给变量 `scan\_table`**。点 Run 时，这段代码会被执行，结果写回状态并刷新下方预览。

作用 它是构建扫描表最灵活的方式：用代码生成任意规则或不规则的扫描点序列 (等间距、对数、随机、网格……)，远比手动敲数字高效。

可用的命名空间 代码执行时，环境里已经为你准备好了：`numpy` (以 `np` 名字提供)、`math` 模块，以及整数 `n\_slots` (当前被绑定的槽数量，等于扫描表应有的列数)。你可以直接用 `np.linspace`、`np.arange`、`np.column\_stack` 等，无需自己 import。

必须满足的契约 代码运行完，命名空间里**必须存在**一个名为 `scan\_table` 的变量，且它是一个二维数组：行数 = 扫描点个数 (你自己定，比如 21、50、100)，列数 = `n\_slots`。第 `j` 列填的就是槽 `s\{j\}` 在它**显示单位**下的数值 (时间槽填 ns、DAC 槽填 -512..+511 的带符号值，0 = 0 V)。

点击 Run 后会发生什么 见下一小节。简单说：代码被执行，`scan\_table` 被读出并写回状态，下方预览随即刷新。

常见坑 (1) 忘了把结果命名成 `scan\_table`，或者拼错变量名；(2) 数组列数和 `n\_slots` 不一致——比如绑了 3 个槽却只给了 2 列；(3) 把 DAC 列写成了纳秒级的大数 (如 50000)，而 DAC 带符号值的合法范围只有 -512..+511；(4) 写成一维数组而忘了它需要是二维 (哪怕只有一个槽，也应是 `N×1` 的形状)。

实用提示 善用 `n\_slots` 让代码自适应：用 `np.column\_stack(...)` 按列拼装，每个槽一列，顺序严格对应顶部说明里的 `s0`, `s1`, ...。先在脑中确认“第几列是哪个物理量”，再动手拼列，能一次写对。

4.6.3 Run 按钮 (绿色)

作用 绿色的 Run 按钮**执行**上方代码编辑器里的 Python 代码。

点击后会发生什么 代码被运行，运行结束后系统从命名空间里取出 `scan\_table` 变量，把它**写回实验状态**，然后刷新下方那张只读的扫描表预览，让你立刻看到生成结果。

接受什么 它不接受参数，就是一个执行触发器；真正的“输入”是你在编辑器里写的代码。

常见坑 如果代码有语法错误、引用了未定义的名字、或最终没有产出合法的 `scan\_table`，Run 就无法把有效结果写回。每次改完代码都要重新点 Run，否则状态里还是上一次的旧表——**只编辑不 Run 等于没生效**。

实用提示 把 Run 当成“编译并预览”这一步：改代码 → Run → 看下方预览的行数和前几行数值是否符合预期，确认无误再去 On Pulse。养成这个习惯能避免拿着错误的扫描表上硬件。

4.6.4 顶部三按钮：Load Program 与两个模板

代码编辑器**上方**有三个按钮，都用来一键**填充编辑器内容**：

Load Program 按钮 作用：从磁盘载入一段 Python 程序 (`.py/.txt`) **到代码编辑器里**。注意它载入的是**代码**，不是数组数据——和载入扫描表数组（在 Delay / Scan 面板的 Load Array）是两回事。常见用途：把一段调好的生成逻辑存成 `.py`，下次直接载入复用。

Template: column_stack 按钮 作用：把**默认模板**填进编辑器——每个槽一列、用 `np.column\_stack` 拼装。它**按当前绑定的槽数自动适配**（一个槽给单列，两个槽给“反相关、总和恒定”的示例）。这是新手最该先点的按钮。

Template: grid 按钮 作用：填入**网格（外积）模板**——用两个数组的 `np.meshgrid` 生成所有组合点，同样按槽数自适应。适合二维扫描（比如同时扫时长和 DAC 码的每个组合）。

载入现成数组在哪

这一页**没有**单独的“Load File”按钮——那会和 Delay / Scan 面板里的 Load Array 重复。要把外部脚本/标定算好的**数组**直接拿来用，去 Edit 页 Delay / Scan 面板点 Load Array，再打开来源开关即可；这一页的三个顶部按钮只管**生成代码**。

4.6.5 Save Array 按钮

作用 Save Array 把**当前的扫描表**保存成一个数组文件，方便归档、复用或在别处分析。

点击后会发生什么 弹出保存对话框，把现在状态里的扫描表（也就是你刚 Run 出来或 Load 进来的那张）写到磁盘。

接受什么 你指定保存路径与文件名；保存出来的就是和扫描表同形状的二维数组文件。

常见坑 它保存的是**当前状态里的表**，所以保存前请确认你已经 Run 过、预览里显示的就是你想要的内容；否则可能把一张旧表或空表存下来。

实用提示 给文件起带含义的名字（比如包含被扫物理量和范围），并和对应的 pulse 配置存在一起，日后回溯“这组数据是怎么扫的”会轻松很多。

4.6.6 下方：只读扫描表预览

作用 编辑器下方是一张**只读**的扫描表预览，按 `s0/s1/...` 列展示扫描表的**前若干行**，并显示总的扫描点数 (point count)。

它告诉你什么 这是你“所写即所得”的核对窗口：列名让你确认每一列对应哪个槽，前几行数值让你检查范围和步长是否正确，点数让你确认这一轮会采多少个点（也就是会发多少次相机触发）。

点击后会发生什么 它**不可编辑**，纯展示。每次 Run、Load Array 或切换来源开关后它会自动刷新。

常见坑 它通常只显示前几行，别因为没看到全部行就以为表被截断了——以显示的**总点数**为准。如果点数或列数和你预期不符，回到代码或文件去查，而不是试图在预览里改。

实用提示 上硬件前，最后扫一眼这里的**点数**和**第一行/最后一行**（如果能看到），快速验证扫描范围的两个端点是否如你设计，这是最省事的“出错前一刻”检查。

4.6.7 完整 workflow：四步走

把整个扫描串起来，一共四步，前面在 Edit 页，后面在 Scan 页和运行：

1. **在 Edit 页绑定字段**：找到你想扫的字段（某周期的 duration、或某周期的 DAC 总线值），点它旁边的**扫描圆点**。第一个绑定的字段成为 `s0`，第二个成为 `s1`，依次类推。绑定后该字段会被高亮标记并显示槽号。
2. **在 Scan 页写代码**：照着顶部“列含义说明”确认每一列是哪个槽、什么单位，然后在代码编辑器里构造一个 `N_points × n_slots` 的数组并赋给 `scan_table`。
3. **点 Run**：执行代码，把 `scan_table` 写回状态，并在下方预览核对行数、列、数值。
4. **On Pulse 运行**：硬件**无缝**地逐行执行这张扫描表，每个扫描点对应一次相机触发。一轮跑完，你就得到了对应所有扫描点的数据。

一句话记忆

Edit 页**绑定**决定有几列 (`s0, s1, ...`)，Scan 页**填数**决定有几行（扫描点）。列错了回 Edit，行错了回代码。

4.6.8 实操配方一：一维 duration 扫描

最常见的需求：固定其它一切，只扫某个周期的持续时间。假设你在 Edit 页只绑定了一个字段（某周期的 duration），它是 `s0`，单位 ns，标称值 `50000`。下面这段代码以标称值为中心、从 `40000` ns 到 `60000` ns 扫 `21` 个等间距点：

Python

```
# 只绑定了一个 duration 槽 s0 (单位 ns)
durations = np.linspace(40000, 60000, 21) # 21 个点, 含两端
scan_table = durations.reshape(-1, 1)      # 形状 (21, 1) = N_points x
↪ n_slots
```

要点 `linspace(a, b, n)` 生成含两端、共 `n` 个等间距值；`reshape(-1, 1)` 把一维数组变成 `N×1` 的二维表（一列对应唯一的槽 `s0`）。

常见坑 别忘了 `reshape`——一维数组不满足“二维 `N×slots`”的契约；时间值用 ns，别误填成别的单位。

提示 想换成对数扫描就把 `np.linspace` 换成 `np.logspace`；想改点数只改第三个参数，预览里的 point count 会立刻反映出来。

4.6.9 实操配方二：扫一个 DAC 电平 (带符号值 -512..+511, 0 = 0 V)

如果你绑定的是一个 DAC 总线值字段 (`[value]` 单位)，它对应带符号 DAC 值 (0 = 0 V)，合法范围是整数 -512 到 +511。假设它是唯一绑定的槽 `s0`，下面在整个量程上扫 32 个点：

Python

```
# 只绑定了一个 DAC 值槽 s0 (带符号值 -512..+511, 0 = 0 V)
codes = np.linspace(-512, 511, 32).round().astype(int)
scan_table = codes.reshape(-1, 1)          # 形状 (32, 1)
```

要点 DAC 列填的是带符号整数，所以用 `round().astype(int)` 取整；范围务必落在 -512..+511 之内。

常见坑 把 DAC 列当成电压或纳秒来填 (写出 50000 这种数) 会超出 +511 的合法范围；浮点值也不合适，记得取整。

提示 只想扫量程的一部分 (比如偏置点附近)，把 `linspace` 的两端改成你关心的码区间即可。

4.6.10 实操配方三：二维扫描 (duration × DAC)

二维扫描就是把两个槽的取值做笛卡尔网格，每个组合是一个扫描点。假设 `s0` 是 duration (ns)，`s1` 是 DAC 带符号值 (-512..+511)。下面构造一个 $11 \times 8 = 88$ 点的网格：

Python

```
# s0 = duration [ns], s1 = DAC value [-512..+511] (0 = 0 V)
durs = np.linspace(40000, 60000, 11)          # 11 个 duration
codes = np.linspace(-512, 511, 8).round().astype(int) # 8 个带符号 DAC 值

# 笛卡尔积: 每个 (duration, code) 组合一行
D, C = np.meshgrid(durs, codes, indexing="ij")
scan_table = np.column_stack([D.ravel(), C.ravel()]) # 形状 (88, 2)
```

要点 `np.meshgrid(..., indexing="ij")` 配 `ravel()` 展平，再用 `np.column\_stack` 按槽序拼成两列——第 0 列是 `s0` (duration)，第 1 列是 `s1` (DAC)。列序必须和顶部说明一致。

常见坑 列放反 (把 DAC 码放进 duration 列) 会让两个物理量错位；网格点数是两边相乘，点数容易爆炸，先在预览里确认 point count 别太大。

提示 想加第三维 (比如再扫一个 DAC 值槽 `s2`)，把它也放进 `meshgrid` 并在 `column\_stack` 里加一列即可。

同时扫多个物理量

DAC 值与 duration 两类槽可以在同一轮里一起扫 (通道延时是固定值，不参与扫描)。你在 Edit 页绑定几个字段，Scan 页就给几列填数，硬件会按行无缝执行这张多列扫描表，每个扫描点照常发一次相机触发。这让“沿某条轨迹同步改变多个参数”或“多维网格扫描”成为一次 On Pulse 就能完成的事。

4.7 Sync 按钮、UNSYNCED 状态与 period 选择

- **Sync (橙色)**：把“sequencer 上实际应用的脉冲”拉回编辑器。当你在 notebook 里用 `PulseController` (`on_pulse/set_channel_delay` 等) 改了设备而 GUI 没跟上时，点一下即恢复一致——服务端在每次成功 prepare 时记录源 payload，GUI 与 API 读的是同一份。
- **UNSYNCED (On Pulse* 星号 + 橙色状态点)**：设备在 RUNNING/PREPARED 状态下，**任何**编辑 (duration/delay/DA/scan/clock/显隐/载入.....) 都会给 On Pulse 按钮加上 confocal 同款的星号并把标题栏状态点变橙——提示“设备上跑的已不是你眼前的参数，按一下应用”。按钮本身保持绿色 (颜色不用于表达状态，避免与 Remove/Load/Sync 的常驻橙色混淆)。若把修改改回原样，星号与橙点自动消失；重新 On Pulse 即清除。Save 的 `Save*` 黄色 + 星号语义表示“未保存到文件”，与运行状态相互独立。
- **点选 period / 中缝**：单击一张 period 卡片 (**整张卡片的外缘**出现一圈圆角描边——只描卡片边缘，不影响卡内任何控件) 或两张卡片之间的空隙 (出现竖线指示)，Add Period 就插到所选位置、Remove 删除所选；再点一次取消选择；无选择时维持旧行为 (加/删最后一个)。扫描 slot 绑定、DAC 总线计划与 repeat bracket 会随中间插入/删除自动重定位。
- **Clear All (标题栏右端, 橙色)**：一键把日程清空——删除所有 period 与所有通道/总线延时，只留下一个 $1\ \mu\text{s}$ 的空白 period (没有任何通道为 on, DA 总线为 Hold 空闲)；扫描绑定、扫描表与 repeat bracket 同时清除。保留的是硬件接线层信息：通道名/语义标签、显示/隐藏状态、DA 总线分组与 clk 指派。清空属于普通编辑：会照常触发 `Save*` (未保存) 与 UNSYNCED 星号，不会自动写盘或上传——从零搭一个新脉冲前先点它最方便。
- **单 period 保护**：只剩一个 period 时 Remove 被禁用 (脉冲表至少要有有一个 period)。
- **拖动排序**：拖动时光标下有半透明卡片缩略图，插入位置竖线为覆盖层 (不引起整行重排抖动)，拖到视口边缘会自动滚动——后面的 period 始终可达。
- **预览横轴自动加宽**：与纵轴的行块一致，每 5 个 period 增加一个基础宽度块，最多 3 倍 (`Live.PULSE_X_PERIODS_PER_BLOCK / PULSE_X_MAX_WIDTH_FACTOR` 可配置)。深度缩放下刻度标签按间距自适应精度，避免退化成一排相同的“8ms”。
- **通道延时**：上限现在约 $\pm 42.9\ \text{s}$ (FPGA 端事件调度器；毫秒级 emCCD 延时直接填入即可)。硬件约束是“该通道任一延时窗口内的在途翻转数 $\leq$ 事件 FIFO 深度 (默认 128, DA 每 bit 64, 可在 `streamer_config.json` 配置)”——编译器对完整日程 (每个扫描点、repeat 回绕缝、跨多帧的长窗口) 做精确计数，稀疏触发信号天然满足，超限会在编译时给出明确报错。延时是**真物理延时**： $\text{out}[t] = \text{in}[t - d]$ 对整条输出流逐 tick 成立，**首帧就是正确的** (不做任何按周期取模的近似)。

5

完整走一遍（新手教程）

本章假设你**从零开始**，一步步搭一个真实的脉冲序列、预览、再加一个扫描。每一步对应上一章讲过的控件，配合 Edit/Preview/Scan 三张截图看。

5.1 第 0 步：打开编辑器

从命令行：

PowerShell

```
.\pulse_gui.bat
```

或在 notebook 里：

Python

```
from Zou_lab_control.frontend.pulse_gui import PulseSequenceEditor
from Zou_lab_control.frontend.qt_fluent import ensure_qt_app
app = ensure_qt_app()
ed = PulseSequenceEditor()          # 空白脉冲表
ed.show(); app.exec_()
```

打开后默认停在 **Edit** 页。顶部三个标签 **Edit / Preview / Scan**，顶部状态条显示可见数、period 数、step、总时长等；状态条右端是 **Clear All** 按钮（一键清空日程到一个 $1\ \mu\text{s}$ 的空白 period，通道名/显隐/总线分组/clock 指派保留——从零开始搭新脉冲时先点它）。

5.2 第 1 步：理解“一段就是一张卡片”

脉冲序列不是网格表，而是一排**周期卡片 (period)**，每张卡是一段持续时间。中间区域横向排着 **Period 1/5**、**Period 2/5**.....每张卡上有这一段的时长、单位、和每条 channel 的勾选框。时间从左到右流动。

5.3 第 2 步：加 / 删一段，设时长

1. 点底部 **Add Period** 增加一段（点 **Remove** 删除当前段）。

2. 在某张卡的 **Duration** 框里填时长，右边下拉选单位 (**ns/us/ms/s**)。例如第一段填 **2 ms** (磁光阱加载)，第二段填 **100 us**。

5.4 第 3 步：让 channel 在某些段为高

在每张卡上，勾选要在这段时间内**打开**的 channel (勾上 = 高电平)。例如让 **trap** 在所有段都勾上、**cooling** 只在加载段勾上、**probe** 只在成像段勾上。左侧 **Channel Names** 列可以给每条 channel 起个好记的名字。

5.5 第 4 步：设延时 (可选)

在 **Delay / Scan** 列，给某条 channel 的延时框填一个值 (可正可负)，用来补偿器件/线缆时延。延时只平移这条 channel 的开关沿，不改变周期长度。

5.6 第 5 步：在 Preview 看波形

点 **Preview** 标签。它画出**未展开**的周期表：每条数字 channel 一行方波，模拟总线一行阶梯线。切 **Show off rows** 开关可显示/隐藏全程为 0 的 channel。这一页只看、不改；改完 Edit 再回来会自动重画。

5.7 第 6 步：把一个字段变成扫描参数

回到 Edit 页，决定要扫什么：

1. 点某个 duration / DAC 值框右边的**扫描圆点** (图 ??)。该字段变淡橙、显示它的 slot 编号 (第一个绑定的是 1)。
2. 想扫第二个参数就再点另一个字段的圆点 (编号 2)，以此类推。

5.8 第 7 步：在 Scan 页填扫描表

点 **Scan** 标签。顶部列出每个 slot 是哪个字段、什么单位。在 Python 代码框里构造一个 **N_points x N_slots** 的数组赋给 **scan_table**，点 **Run**：

Scan 页：扫一个 duration

```
import numpy as np
# 单 slot: 把绑定的那个时长从 1 us 扫到 10 us, 共 10 个点。
scan_table = np.linspace(1000, 10000, 10).reshape(-1, 1) # 单位 ns
```

也可以在 Delay / Scan 面板点 **Load Array** 从 **.numpy/.csv/.txt** 直接载入现成数组 (再用来来源开关启用)，或在本页点 **Save Array** 把生成的表存出来。

5.9 第 8 步：运行 / 保存

1. 回 Edit 页，点 **On Pulse**：它先把当前脉冲编译上传到 sequencer，再启动。硬件会无缝地逐个扫描点跑，每点触发一次相机。
2. **Stop Pulse** 让输出回到安全态。
3. **Save** 把脉冲 **.json** + 预览 **.png** + (有扫描表时) 扫描数组一起存出，三件配套。

5.10 完整例子：扫一个 DAC (DA) 电平

把上面串起来，做实验里最常用的“扫 DA”：

1. Edit 页搭好脉冲；在某一段把某条模拟总线（如 `da_dipole`）的 DAC 值框的扫描圆点点亮。
2. Scan 页填**带符号 10-bit DAC 值**（-512..+511，0 = 0 V；不是电压、不是 ns）：

Scan 页：扫 DAC 码

```
import numpy as np
scan_table = np.linspace(0, 1000, 11).astype(int).reshape(-1, 1)
```

3. Run 写回，回 Edit 页 **On Pulse**。FPGA 逐点切 DAC，硬件无缝（无需每点重传）。在 Preview 里，被扫的 DAC 段会高亮显示在**它那条总线的行上**并标着 slot 编号。

5.11 遇到问题先看哪里

Preview 不出图 切一下 **Show off rows** 触发重画；确认脉冲表非空、最后一段把所有 channel 归零。

扫描报错 确认每个时间扫描点是 20 ns 的整数倍；DAC 列填带符号码（-512.. + 511）。duration 与 DAC 可以同时扫（仿射总线 tick），报错信息会指出具体哪一列哪个点不合法。

On Pulse 无信号 见主手册“真实硬件 Runbook / 常见问题”。

6

Task 控制台（实验仪表盘）

6.1 它是什么

Task 控制台 (`task\_console.bat / frontend.show\_task\_console`) 是实验侧的可配置仪表盘：一个网格里摆任意多个**实时面板**，每个面板用一个**信号名或一小段表达式**接到实验数据上。把布局设计好之后**按名字保存成一个“任务仪表盘”**（如 `atom\_loading\_monitor`），以后一条命令就能整套载回——实验室里每类日常实验配一套自己的监控台。

数据通路分五层（device / measurement / processor / task / plot），互相只通过**信号集线器 (SignalHub)** 耦合：

SignalHub (`neutral\_atom/core/signals.py`) 线程安全的命名信号环：实验循环每个 shot `publish` 一批命名值（标量或数组），控制台轮询 `latest/history`，用版本号跳过无新数据的刷新。

逻辑节点 (`neutral\_atom/operations/logic.py`) 三类生产节点共用 `LogicNode` 基类（同一 worker-loop + 协作取消 + owner-thread 参数队列）。`Measurement` 驱动设备采集并发命名信号（相机 `CameraMeasurement` 发 `frame`、扫描型 `ScannedMeasurementNode` 让有限扫描在 Monitor 里逐点长出 live 曲线、自停）；`Processor` 是无采集的**反应式**变换节点（“func”层），消费 hub 信号、republish 派生信号，如 `OccupancyProcessor` 用与真机读出**相同的** `calibration.detect` 契约把 `frame` 变成 `counts`（逐站计数）/`occupied`（占据）/`rate`（滚动装载率）/`rate\_sites/rate\_grid/centers/thresholds`；`Task` 是一次性编排（如 `CalibrateReadoutTask` 产 `TrapCalibration+npz` 产物、中途把模板帧流到一个面板）。**装载读出由你用这三类节点自己拼**（标定 task + 相机 measurement + detect processor），**没有单体节点一次造出所有信号**——每个节点只经 `CameraDevice` 契约取帧，换真机只换传入的 `camera`（唯一虚拟之处 = 相机数据源）。所有节点都可同步驱动（`step()`，测试/Notebook）或后台线程（`start(rate\_hz=...)`）。

面板 (panel) 即 frontend live 绘图类的复用，所有图内交互（滚轮缩放、中键平移、右键十字、框选、拖 clim/阈值线）原样可用。plot 面板是**纯视图**，只订阅信号名，与采集**解耦**（加一张 plot 不启动任何东西，见下）。

6.2 六种面板类型

2D image (Live2DDis) 任意二维数组（相机帧、装载率网格、两台实验之差）。自带右侧分布与 colorbar，可在分布带上拖动 clim 上下限。

Site map (LiveSiteMap) **占据叠加图**：一张实时相机帧（灰度 2D + 计数 colorbar）上，每个阱位画一个**空心圆环**——空位是浅色半透明的细环、有原子是深色不透明的粗环（Rb87 读出风格，配色单一来源 `style.SITE\_OCCUPANCY\_STYLE`）。占据由 `data\_y` (≥ 0.5 判为有原子) 决定，圆环不填充以透出底帧；站点中心来自一个 `centers` 信号 ($(N, 2)$ 相机像素，真机即 `TrapCalibration.centers`)。它是“普通 2D + 占据圆环”而非热图，所以没有 colormap——唯一的 colorbar 是相机帧自己的计数；这条相机帧 clim 和 2D image 一样吃 `relim` (`tight/normal/fixed`)，所以“改 lim / 钉 fixed lo-hi”对 Site map 同样生效（包括 Edit 标签里那张冻结快照）。几何与 2D 面板逐像素对齐。

Distribution (HistogramFigure) 直方图 + 自动双高斯拟合 + 可拖阈值线 + 保真度文本——把 `history('counts', 200).ravel()` 接进来就是读出阈值的实时体检。

Rolling trace (LiveLiveDis) 标量滚动轨迹 + 右侧分布（直方图 + 高斯 σ 拟合），图内右上角常显最新值。保真度、温度等“一条曲线盯一个数且关心其分布”的场景用它。

Rolling trace (no dist) (LiveLive) **同样的滚动轨迹但去掉右侧分布带**——对装载率这类**右侧直方图/ σ 拟合没有物理意义**的量，用这个变体（整条轨迹占满宽度，仍显最新值）。它是**独立的 plot type**：带分布的 `LiveLiveDis` 反过来**继承**它、在其上**加**分布带，而不是在一个类上切布尔开关。装载率这类量：Add 一个这种 plot 面板、把 Source 指到某 processor 发的 `rate` 信号即可（控制台开局空，布局由你自己拼 + Save，无内置预设）。

1D vector (Live1D) 一维向量逐点连线（逐站装载率、逐站计数……横轴是站号）。

6.3 Setting: 每个面板的基本配置

卡片标题行右端的 **Setting** 文字按钮打开该面板的设置弹窗。排版照搬 `confocal_gui` 的**竖排**范式：粗体小标题占独立一行，下方每行 **[固定宽度标签 | 控件]**（复用 `FluentSectionLabel` 与 `FluentSettingRow`，全 frontend 共享同一节律）。从上到下五段——**Source / Display / Unit / Limits / Panel**，全部即时生效（只 Source 因要校验表达式有自己的 Apply，其他控件改完就立刻刷新面板）。命令行拟合、`relim` 这类重控件仍在 **Control** 标签页（**Edit...** 按钮进入）：

Source (接什么数据) `signal` 下拉列出 hub 里**此刻真实存在的所有信号名**——选一个，面板立即切到 `value = 信号名`，不用打字。要做运算（差值、切片、历史堆叠）就在下面的表达式框写一行 Python 并点 **Apply**（或回车）；下拉会自动回显纯信号表达式。命名空间：每个信号名（最新值）、`history(name, n)`、`latest(name)`、`names()`、`shot`、`np`、`math`。表达式出错时 **Setting** 按钮变红，错误只落在**该面板自己的**状态行，其余面板照常刷新。

Display (怎么显示) `size` 尺寸预设；2D / Site map 的 `colormap`——**这就是改 colorbar 配色 (colorset) 的地方**（没有“显隐 colorbar”开关，因为带 colorbar 的图恒显 colorbar）；以及各类型的**声明式参数**（`PANEL\_PARAMS` 里一行一个 `ParamDecl`，控件由 `param\_widgets.PARAM\_WIDGETS` 注册表按 kind 自动生成）：Site map 的 `centers/image` 信号名（`image` 留空 = 不垫相机帧）、Rolling trace 的历史长度、Distribution 的 bin 数。

relim 与 Unit Display 段最后两行是**轴控制**：`relim` 下拉用 `confocal_gui` 的 `combo\_relim` 命名 `tight`（紧贴数据自适应）/ `normal`（带边距自适应）/ `fixed`（**钉死**上下限——选 `fixed` 时下方多出一行填 lo/hi），写进 `config.params["relim"]`——这与该面板 Edit 标签里的 `relim`

控件是**同一个 key** (单一真相源, 绝不漂移), 并对 **2D image 与 Site map 的 colorbar clim 同样生效** (相机帧颜色范围跟着 tight/normal/fixed 走, 不只是 1D 的 y 轴); **Unit 按钮** + 右侧当前 unit 倍数。点一次在该轴单位环上前进一档 (GHz/nm/MHz 或 ns/us/ms 等, 复用 `DataFigure.change\_unit`), 轴标签无定义单位时是 no-op; 当前 `unit\_index` **随布局 JSON 一起保存**, 重建时由 `\_apply\_display\_params` 重新应用——所以“这张卡我切到 normal + us”保存/载入/重建后都保持。**Setting 没有手动 x/y 轴 lim 四框** (confocal_gui 也没有, 那在 **Edit...** 标签)——但把**数据范围数字钉死**就用上面 `relim=fixed + lo/hi` (**就在这儿, 不必去 Edit**); 坐标轴范围细看用画布缩放 / 拖拽。

Panel (这张卡) 标题(显示在图内, 改动通过 `BaseLivePlot.\_apply\_title` → `style.apply\_title` 走密封 API, 字号永远是 `title\_fontsize()`); 一行操作 **Remove** (移除)、**Edit...** (打开该面板自己的 Edit 标签做命令行拟合 / 手动 lim / 详细处理, 导出图的 **Save Fig** 也在那里)。

一张**满分辨率相机帧** (如 1920×1200) 怎么显示到面板上? 不是把整幅原图交给 matplotlib 每帧重采样 (那样又慢又糊), 而是走下图这条链: 全分辨率数组**留在内存**, 显示层按**当前视野裁剪**、再**块平均**降到该 axes 的像素预算 (面积平均, 比最近邻抽样**提信噪**), 交给 `imshow(interpolation="auto")`; 嵌入画布把 Agg 缓冲渲染成**恰好等于屏幕设备像素**的大小做 1:1 贴图。整条链**只有一次重采样**; 放大时从满分辨率数组**重新切片**, 细节随之**渐进回归**, 直到视野内 1:1。

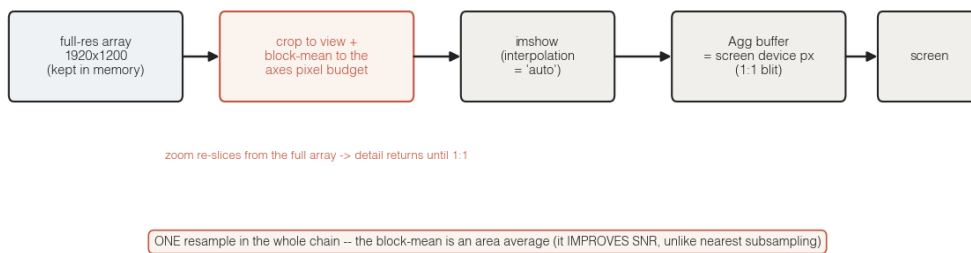


Figure 6.1: **2D live 图像的显示链**。满分辨率数组留在内存; 显示层按当前视野裁剪 + 块平均到 axes 像素预算 (**面积平均, 提信噪**), `imshow(interpolation="antialiased")` 渲染, Agg 缓冲**等于屏幕设备像素**做 1:1 贴图。整条链**只一次重采样**; 放大重新切片、细节渐进回归到 1:1。这就是为什么 monitor 相机既不卡、又不比原图糊。

Note

弹窗用 `FluentPopup (qt\_fluent.py)`: **半透明、无边框**的顶层 `Qt.Popup`, 圆角白卡由 `paintEvent` 抗锯齿描一笔 (填充 + 1px 边框)。两条教训: □ 不可在**不透明**顶层弹窗上用样式表 `border-radius`——方形窗体留在圆弧外, 右下角露出方角白块 (那条“奇怪的线”), Windows 还叠一层原生阴影; □ 光加 `WA\_TranslucentBackground` 不够, 它会让样式表背景**整块不画** (弹窗变透明)。所以圆角卡**自己画**。边框是画出来的、不是样式表规则, 因此也**绝不会级联**到 `QLabel/QComboBox` 等子控件。

常用表达式示例:

Python

```

value = frame                                # 最新相机帧 -> 2D
value = occupied                              # 每站 0/1 占据(Site map 设
↪ repeat_mode=average=逐站装载概率)
  
```

```
value = history('counts', 200).ravel() # 最近 200 shot 计数 -> Distribution
value = occupied - b_occupied           # 两个实验相减 -> 1D
```

6.4 布局：拖动吸附的模数化卡片

面板尺寸是**有限的预设组合** (`frontend.PANEL\_SIZES`: 1x2 / 2x2 / 1x4 / 2x4, 行 × 列的“半单元”数)。设计规则与 confocal 一致：**所有面板类型共享同一块 plot 区域和同一组边距**——2x2 就是 frontend 的标准 plot 区域 (数据区 480 × 360), 1x2 是它的半高, 2x4 是双宽。2D 与 Site map 在**自己的区域内部**做 [0.75, 0.1, 0.1] 水平分割, 所以 2x2 的主图恰好 360 × 360 **正方形**且与 colorbar 顶底对齐——不同类型面板因此能整齐排版。接口只有 `panel\_plot(data, kind=..., size=...)` 与 `panel\_plot\_spec(size)` (`frontend/live.py`): **尺寸是唯一旋钮, 几何与美术不对外暴露**。

每个面板是一张带 1 px 扁平边框的标题卡片 (fluent 卡片设计), 卡框标题 = 面板类型, 画布下方、卡片底部一行灰色状态行 (当前 shot 与来源表达式回显)。排版不用填行列数字: **按住卡片边框 (画布以外的任何空白处) 拖动**, 松手时卡片自动**吸附到落位判据**。落位规则很直接 (见下图): 以被拖卡的**左上角点**为判据, 对**每张已存卡的左上角**与**每个邻近空缺格点**比最近距离——离某已存卡的角**近就挤走它** (那张卡随即被重力排到下方让位), 离某空缺格点近就**落到那个点**; 之后照旧走**西北 (左上) 重力压实** (`\_compact`)——每张卡在自己的列内**向上落到上方卡或顶端为止**、被邻居挡住就停下**绝不越过障碍传送到远处空位**, 于是结果**既不重叠、也不留竖向空隙**。卡片外框恰好等于整数个布局槽 (卡框状态行区吸收图边距的模数差额), 所以**一张 2 × 2 卡严格等于两张 1 × 2 卡叠起来加一道间隙**, 任意尺寸混排严丝合缝。标题栏: **Add Panel** 按所选类型加面板、**Save / Load** 存取布局 JSON; 面板值形状变化 (换表达式/尺寸/参数) 自动重建该面板。

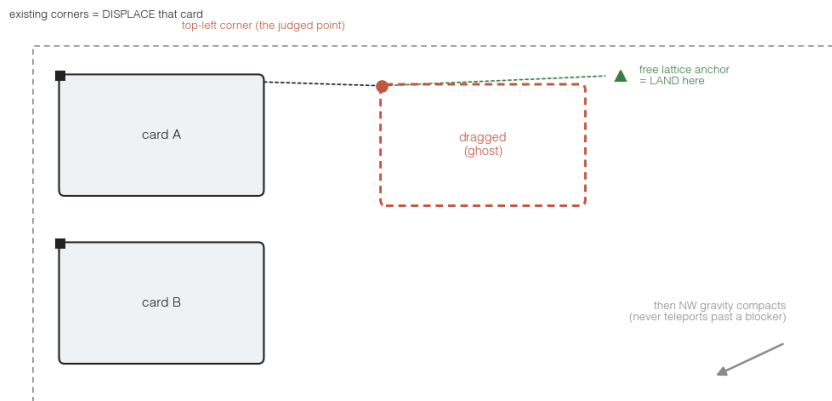


Figure 6.2: **拖动落位规则**。松手时以被拖卡的**左上角点** (红点) 为判据, 对两组候选比平方欧氏最近距离: **每张已存卡的左上角** (黑方块) ——赢 = **挤走**那张卡 (它被重力排到下方让位); **每个邻近空缺格点** (绿三角) ——赢 = **落到该点**。定完落点再走西北重力压实: 卡只向上/左落、被邻居或边界挡住就停, **绝不越过障碍传送到远处空位**。所以你把一张卡拖到另一张上方就是“占它的位、把它挤下去”, 拖到空地就是“停在最近的整齐槽位”——两种意图各由一组候选覆盖。

6.5 两个常驻标签 + 每面板/每节点的 Edit 标签

窗口有两个常驻 (不可关) 标签: **Monitor** (plot 面板的实时拖拽网格) 和 **Logic** (measurement / processor / task 逻辑节点的列表, 每行一个状态点 + Edit) ——**没有空的全局 Control 标签**。plot 面板各有自己的 Edit 标签 (点面板的 `Edit...`), **logic 节点**各有自己的 Edit 标签 (点 Logic 行的 `Edit...`):

标签栏多开一个**可关闭**的标签 (右上一个**柔和的 × 关闭**键, 灰色、悬停才高亮, 不是刺眼红叉; 多个可并排各开一个, 关掉即销毁)。重控件全在这里, 绝不压垮 Setting 弹窗。

6.5.1 从 Add Panel 建节点: 按架构层全暴露

连了会话 (`show\_task\_console` 传 `session= + measurements=/processors=/tasks=; -config /` 虚拟启动都自动传) 后, 标题栏的 `Add Panel` 下拉把**五层架构里每个可加的节点**按层列出, 而不只是 6 种 plot kind:

- **Plot: X**——一张**纯视图**, 连一个 hub 信号 (`value = <key>`);
- **Measurement: Camera (live frames)**——**相机持续流**(一等可加的 measurement 节点, 独立于 loading 读出), 其 Edit 就是相机自己的 `exposure/region`;
- **Measurement: < 扫描名 >**——自动发现的扫描测量 (温度、读出保真度...);
- **Processor: < 名 >**——“func” 层变换节点 (逐帧 detect、逐站保真度...);
- **Task: < 名 >**——一次性编排 (如 `Calibrate readout`), Edit = 参数 + **Run**。task **不向 hub publish**: 结果/标定留在节点上, 中途模板帧 + 进度写进它自己的缓冲 (`node.output`)。Run 时控制台在 Monitor 自动开一张**固定**面板显示它跑时的过程 (读保留键 `\\_task\_frame\_`, 不经 hub), 并以橙色横幅**锁定其他一切操作只留 Stop** (confocal task 式); 完成/Stop 后解锁并撤掉该瞬态面板。

装载读出**没有一键复合体**: 自己加 `Measurement: Camera (live frames) + Processor: Judge occupancy + Task: Calibrate readout` 三个节点拼出来——每层独立、信号显式相连。

`measurement / processor / task` 三份目录都是**自动发现的**: 往 `operations/measurements/、operations/processors/、operations/tasks/` 丢一个带 `@measurement/@processor/@task` 装饰的工厂模块, **下次开 task_console 它就在对应下拉组里**, 无需改任何 list (临时用 `register\_measurement/register\_processor/register\_task` 选一个再 `Add Panel`, 就在 Monitor 板建它的节点 + 面板**并打开它的 Edit 标签**——节点的参数表单 + Start/Run 就在那里。**没有单独的全局发起器**。

两类 Edit 各属一套: plot 面板的 Edit 有曲线 fit + 手动 lim + Acquisition (它所读信号的**产出节点**自报的源参数 + Apply), **但绝不内嵌某测量的“参数表单 + Start”**——那在该 logic 节点自己 (Logic tab) 的 Edit 里; logic 节点 (measurement / processor / task) 的 Edit 只有该节点自己的参数表单 + Start/Stop (**无 fit**——拟合是 plotter 的事, 去 Plot 面板做), task 的 Start 即 **Run**。面板底部 footer 的信号图例用**层名**显示信号来路 (如 `from camera [measurement]、from camera > detect [processor]`), 绝不漏 Python 类名。

6.5.2 Edit 标签 (PanelEditor) 的内容

整页**可滚动** (快照再大也不会把底部控件挤出可视区)。内容**不与 Setting 重叠** (Setting 只留源 / 尺寸 / colormap / relim / unit):

Acquisition (编辑数据源的采集参数) 一张 plot 面板是**视图**, 它读的信号由某个**逻辑节点**生产, 背后是**数据源**——相机本身, 或一个变换节点。Edit 顶部列出该**数据源经 acquisition_parameters() 自报的可编辑参数**: 一张原始相机帧面板 (源是 `CameraMeasurement`) 列出相机的 `exposure / region` (ROI 端点); 一张占据面板 (源是 `OccupancyProcessor`) 列出它的 detect 设置。每格预填**当前正在跑的值**并在下方标 **now: < 值 >** 供改参时参照; 改完点 **Apply** 就把改动**原地**推给数据源 (相机即 `configure` 实时重配, **不重启节点**——节点从它自己在 Logic tab 的 Edit

Start), 活面板以同一信号名无缝续刷。**所以一张 qCMOS live 2D 的 Edit 就能直接调相机曝光/ROI。**(plot 面板**绝不**自带某 measurement 的“参数表单 + Start”, 那在 Logic 节点自己的 Edit 里——见下。)

Parameters (plot 的 API 参数) 该面板 **plot 调用里可变的参数**自动铺成 GUI 控件 (`length / bins / centers / image.....`, 由该 kind 的声明**自动发现**); 改一下就**实时重渲染**活面板与快照。`colormap / relim / unit` 这类**轻量显示旋钮** Setting 与 Edit 的 Display 段**两处都给** (同 **key 单源**、改一处另一处跟着, 不是两份逻辑), 其余 `display=False` 的功能参数只在这里。

框选写回源的采集参数 (而非内嵌测量表单) plot 面板**不内嵌**某测量的“参数表单 + Start”——那在该 measurement 作为 Logic 节点时它**自己的** Edit 里 (Logic tab, 见上)。这里保留的是**框选写回**: 在 Edit 快照上拖一个矩形 (或缩放/平移), `\_read\_region` 把端点 (`x\_min, x\_max, y\_min, y\_max`) 交给产这张图的**源节点**的 `region\_to\_acquisition\_parameters`, 源各转各的格式 (相机节点 → ROI 矩形、二维扫描 → 两条轴范围), **填进上面 Acquisition 段的对应输入框**, 点 Apply 生效 (学 `confocal\_gui` 的 `area.callback`; 缩放/平移与框选共用同一回调, 框选 OVERRIDE)。填的是**采集参数**本身、不是坐标轴 lim。

Fit (全套 data_figure) 模型下拉是**完整的 DataFigure 集合** (Lorentzian / Gaussian / Lorentzian (Zeeman) / Rabi / Exp decay / **2D center**), **不按 plot 类型门控**——2D 图用 2D 高斯 `center` 拟合, 其余拟合 1D 轨迹。一行自由文本拟合参数 (`p0=[...]` 初值、**名字 = 值** 固定、`is\_fit=False` 只叠加不优化) + `Fit/Clear`; 底层 DataFigure (与 notebook 的 `p.data\_figure.lorent()` 同源)。

手动 x/y lim + 冻结快照 + Save Fig 四个数字框 + `Apply lim` (调 `DataFigure.xlim/ylim`, 作用在快照上, 打开时自动回填); 快照**照搬 Monitor 那张卡的 size** (与活面板**同尺寸**、绝不裁切、也绝不强塞一个固定 2x4 撑破窄视区), 画布**钉成固定大小** (`pin\_size: min=max=` 设计尺寸, 连画布自己 deferred 的 `resync` 也守住), 所以**改一个参数重抓快照时它既不会胀大、也不会缩没**; 重抓走**先建后换** (新图建成功才换, 建图报错就保留上一张好图、绝不留空白), 并**每种 kind 含 Site map 都吃 relim/fixed** (由单一 `\_view\_kwargs` 同时喂活面板与快照, 所以 `sitemap` 改 `lim` 也随之刷新)。配 `Refresh` 重抓 + `Save Fig` (一个 `png/pdf/jpg` 格式下拉 + 按钮, 导出**当前面板**的图 + 同名 `.npz` 到 `tasks/`, `npz` 数据不随图片格式变)。这个 Save 内部先把图 `bind\_source` 到该面板接的信号、再调 `DataFigure.save`——**与 notebook 的 `p.save()` 是同一套核心** (见“保存与回看图”一章), 所以 GUI 面板 Save 与 notebook 存图产生的 `npz` 逐字一致, 都能用 `exp.figure\_viewer` 回看。

选择器分工: Monitor 卡**默认停用所有选择器** (工具已就位但未武装 + `isolate\_wheel=False`) ——滚轮直接**滚仪表盘**、绝不被框选/缩放吞掉; 表头的 **Selectors** 开关可就地武装全部卡片 (框选/十字/缩放/拖线即刻可用, 滚轮转为图内缩放), 关掉立即回到显示专用。Edit 快照**始终带完整选择器**。每个 Edit 标签是**独立快照**, 不打扰 Monitor 实时刷新; `relim / unit` 这类只在 Setting, 避免重复。

6.6 保存 / 载入布局: 设计一次, 随处载回

布局 (面板、位置、尺寸、表达式、参数 + Logic 节点声明) 整包存成**一个 JSON**, 文件名就是任务名。**没有内置预设**——唯一的“任务”就是**你自己 Save 出来的布局**。三个入口完全等价:

- **GUI**: 标题栏 `Save` 默认存到 `tasks/< 任务名 >.json` (任务名即左上的名字框); `Load` 选一个 JSON 整套载回。`Save*` 黄星 = 有未保存改动 (与脉冲编辑器同语义)。**没有 Presets 下拉、没有内置预设**——唯一的“任务”就是你自己 Save 出来的布局。
- **命令行**: `task\_console.bat -task < 你存的名字 >` (读 `tasks/< 名字 >.json` 或一个 JSON 路径)。

- **Notebook:** `zf.show\_task\_console(hub=hub, task="< 你存的名字 >")`。

配置一套新监控台的完整流程：开一个空控制台 → 从 Add Panel 加 Logic 节点（相机 measurement / 扫描测量 / processor / task）并 Start、加 Plot 面板把 Source 指到它们发布的信号 → 改任务名 → Save。以后一条命令整套载回——实验室里每类日常实验配一套自己的监控台。

6.7 Devices 按钮：只读查看当前设备

标题栏的 **Devices** 按钮打开一个**只读**的设备查看器：每台已加载设备一个 tab，显示它的 snapshot + live 运行时读回（就是设备管理器 Control tab 那份 `runtime\_controls()` 目录的只读形态），**没有配置编辑、没有增删设备、没有 Apply 写**。这样操作者能在实验**正跑着**时安全地瞄一眼某台设备的状态（曝光、扫描进度、firing.....），而**不会误改跑在其上的设备**。要真正编辑 / 换设备，走**完整的**设备管理器 `exp.device\_manager()` / `na.device\_manager()`（它才有 Config 编辑器与 Load / Apply；换设备会先停用被换设备的节点——详见设备手册）。查看器每会话单例，重开复用同一窗口并**每次重读当前设备**（`load\_config` 换过设备后也能看到新的那套）。

6.8 启动与接线

双击 `task\_console.bat`（或 `python task\_console.py`）即以**虚拟实验**启动：走与**真机**同一个 **connect()** 契约连一个虚拟实验（只有相机帧是仿真的），`sitemap/thresholds` 自标定让目录里的 measurement / processor / task 能跑，再把**自动发现的三份目录接进 Add Panel**——但控制台开局 **空 hub、全停，不自动建/启任何读出**（解耦的 VIEW / LOGIC 模型）。看 loading 你自己从 Logic tab 加 Measurement: Camera (live frames) + Processor: Judge occupancy + Task: Calibrate readout 并 Start，再加 Plot 面板连它们的信号。常用参数：`-config remote\_template`（**真机直跑**：连真实设备、同一 `connect()` 契约——首光站点/阈值标定仍在 notebook 做，见 `docs/REAL\_HARDWARE\_BRINGUP\_zh.md`）、`-grid 5x7`（阵列形状，虚拟与 `-config` 通用）、`-task 名字`（载入你存的命名布局）、`-state xxx.json`（按路径载入布局）、`-scale`（UI 缩放，默认 auto）、`-no-connect`（不连任何会话，自己接 hub）。

从 Notebook 接**真实实验**：建一个 hub，开控制台（开局空——自己加 Plot 面板把 Source 指到下面 publish 的信号，或 `task=" 你存的布局"` 整套载回），在实验循环里每个 shot publish 一次——发布的信号集尽量沿用约定信号名（`frame/counts/occupied/centers/rate.....`），这样同一套命名布局在虚拟和真机之间**零修改通用**：

Python

```
from Zou_lab_control import frontend as zf
from Zou_lab_control.neutral_atom.core.signals import SignalHub
import Zou_lab_control.neutral_atom as na

hub = SignalHub()
console = zf.show_task_console(hub=hub)    # 开局空；自己加 Plot 面板把 Source
↳ 指到下面 publish 的信号

exp = na.connect("virtual")                # 真实实验换成对应 config
centers = exp.readout.sitemap(frames=12,
↳ display=False).calibration.centers
rate = None
for shot in range(1000):
    frame = na.triggered_frames(           # 唯一的 arm-before-fire 编排：先 arm
↳ 相机再 fire
```

```
exp.devices.camera, exp.devices.sequencer, exp.sequence, 1)[0]
det = exp.readout.from_image(frame, display=False)
occ = det.occupied.astype(float)
rate = occ.mean() if rate is None else 0.95 * rate + 0.05 * occ.mean()
hub.publish({"frame": frame, "counts": det.counts, "occupied": occ,
            "centers": centers, "rate": rate, "shot": float(shot)})
```

读出保真度一类的慢量也一样：算出来 `hub.publish({"fidelity": f})`，再开一个 Rolling trace 面板把 Source 指到 `fidelity` 即可。

6.9 给维护者：扩展面板

给某类面板**加一个参数**= 在 `frontend/task\_console.py` 的 `PANEL\_PARAMS` 里加一行 `ParamDecl(key=, label=, kind=, default=, ...)` (`kind` 选 `choice/int/signal` 等, `display=True` 进 Setting、`False` 进 Edit) , Setting 控件、JSON 保存与图重建全部自动跟上; 在对应的 `\_build\_plot` 分支里消费 `config.params[键]`。**加一种参数控件类型**= 在 `frontend/param\_widgets.py` 写一个 `ParamWidgetHandler` 子类并进 `PARAM\_WIDGETS` 注册表 + 在 `ParamDecl` 的 `kind` 白名单加一项——测量表单、Setting、Edit 三处一齐生效 (`test\_param\_widget\_registry` 机械守卫)。**加一种新面板类型**: 在 `frontend/live.py` 写一个 `BaseLivePlot` 子类并进 `plot()` 工厂的 `kind` 分发 (几何必须走 `panel\_plot\_spec` 的共享区域与边距), 然后在 `PANEL\_KINDS/PANEL\_PARAMS` 各加一项、`\_coerce/\_render/\_build\_plot` 各加一个分支。可视化改动必须用 `devtools.capture\_user\_view` 三档缩放截图验收 (`parity` 目标同屏对比两个 GUI 的控件尺寸)。

7

对外接口完整参考

读完这一章，你应当能在 notebook 里**自信地调用每一个公共函数并预期到正确结果**。每个接口按统一六段式写：**一句话用途 / 完整签名 / 每个参数 / 数组（数据）契约 / 可运行示例 / 不可配置的边界**。约定 `from Zou_lab_control import frontend as zf`，全章用 `zf` 前缀。

7.1 先读：密封 API 的设计契约

`frontend` 对外暴露一个**小而密封**的接口。外部调用者只传**数据 + 少量语义选项** (`labels`、`kind`、`size`、`bins`、`thresholds`.....)，拿回**正确的美术、几何、dpi、字体。几何、边距、dpi、配色、阴影、缩放规则**由 `frontend` 拥有，**不对外开放**。

Note

为什么几何不是旋钮？（这样你就不会困惑 “我想改边距怎么没有参数”）

- 全项目只有一套 **300 dpi 排版系统**和一条 **auto-scale 规则**：外部乱改会在不同 Windows DPI 缩放下破坏对齐契约（典型后果：两个 GUI 控件大小不一致、高 DPI 下图变形）。
- 所有面板**共享同一块 plot 区域和同一组边距**，才能让任意类型/尺寸的面板严丝合缝拼接（`confocal` 模数规则）。
- 2D / Site map 图**恒为 square**且与 `colorbar` 对齐，这是设计的一部分，不是可关掉的开关。

你能配置 (DATA)	你不能配置 (由 frontend 拥有)
数据数组、 <code>labels</code> 、 <code>title</code>	<code>data_px</code> / <code>margins_px</code> / <code>spec</code> (几何)
<code>kind</code> 、 <code>size</code> (从预设里选)	<code>dpi</code> 、字号、字体
<code>bins</code> / <code>thresholds</code> / <code>cmap</code>	<code>bad_color</code> / 调色板 / 阴影
<code>update</code> 模式、是否 <code>display</code>	缩放系数 (<code>scale</code> 仅 <code>None</code> = 自动)

试图传被密封的参数会**直接报 `TypeError`**（不是被悄悄忽略），错误信息会告诉你该用什么。权威定义见源码 `frontend/_\__init___.py` 模块文档（六条规则）与 `frontend/AGENTS.md`。

7.2 zf.plot() ——绘图工厂（最常用）

一句话用途：从统一数组契约一次性创建静态或实时 notebook 图，自动选图型、自动套 300 dpi 排版。

完整签名：

Python

```
zf.plot(data_x, data_y=None, *, kind="auto", update="once", labels=None,
        display=True, data_figure=True, watch_interval=None,
        stop_when_full=True, done=None, points_done=None,
        copy=False, lock=None, **kwargs)
```

每个参数：

data_x / data_y 见下方数组契约。

kind 图型：**auto**（按 coord_dim 自动选 1d/2d）、**1d**（折线，别名 line/trace）、**2d**（图像，别名 image/map）、**hist**（直方图，别名 distribution）、**monitor**（滚动轨迹 + 侧分布，别名 rolling）、**sites**（原子站点图，别名 site-map/sitemap）、**pulse**（静态时序图，别名 sequence/timing）。
注意：kind="pulse" 不支持 update="watch"（会报错）。

update **once**（默认，静态）或 **watch**（启动前端定时器，从共享数组刷新）。watch 模型：采集线程在另一侧**原地修改**数组，前端拥有 artist 与刷新。

labels (xlabel, ylabel, zlabel)；各 kind 有不同默认(1d/2d=(X,Y,Z)、hist=(Counts,Shots,Population)、pulse=(Time (s), 空, State))。

display 是否立刻在 notebook 显示。

data_figure 是否附 DataFigure 句柄（用于 .lorent()/ .decay() 等拟合）。

watch_interval / stop_when_full / done / points_done / copy / lock watch 模式的刷新间隔、满即停、完成回调/标志、是否拷贝、线程锁。**线程契约：**阻塞式硬件采集不能跑在 UI 线程里。

****kwargs** 透传给具体 plotter 的**数据**选项：cmap、bins、thresholds、reim_mode、title**这些是数据选项，不是几何选项**。传 data_px/margins_px/spec/dpi/bad_color 会报 TypeError。

数组契约：data_x 是 (N, coord_dim)（一维自动升维）；data_y 是 (N, channel_dim)，长度须等于 N；coord_dim=1 画一维线图、=2 画二维扫描图；kind="hist" 把 data_x 当数值数组；未完成的点用 NaN（前端据此推断进度）。

可运行示例：

Python

```
import numpy as np
from Zou_lab_control import frontend as zf

# 一维 + 洛伦兹拟合
x = np.linspace(737.0, 737.2, 220).reshape(-1, 1)
y = (lorentzian(x) + noise).reshape(-1, 1)
p = zf.plot(x, y, labels=("Wavelength (nm)", "Counts/0.1s", "Counts"))
p.data_figure.lorent(is_display=True)
```

```
# 二维扫描: data_x=(N,2)
zf.plot(coords_xy, signal.reshape(-1, 1), kind="2d", labels=("X", "Y",
↪ "Counts"))
```

不可配置的边界: 2D 图恒为 `square` (传 `square=False` 会报错; 要非方形请直接用内部类 `Live2DDis`); `dpi/边距/字号` 不可传, 由 `FigureSpec` 内部决定。

7.3 各图型速查 (plot(kind=...) 的七个面孔)

这些 `plotter` 类都公共导出、可直接构造, 但**首选** `plot()`:

1d -> `zf.Live1D` 一维折线。横轴默认是站号; 也支持 `(N,2)` **x-y**: `data` 传 `(N,2)` 数组时 `col0=x`, `col1=y` (这是 Task 控制台 Measurement 结果曲线把真实 `t_off` / detection time 当横轴的方式)。

2d -> `zf.Live2DDis` 图像 + 右侧分布 + colorbar + 可拖 `clim` (`cmap` 选配色)。

hist -> `zf.HistogramFigure` 直方图 + 自动双高斯拟合 + 可拖阈值线 (`bins/thresholds`, 方法 `classify/fractions`)。

monitor -> `zf.LiveLiveDis` 标量滚动轨迹 + 侧分布(直方图 + σ 拟合)。别名 `rolling/loading\_rate` 都归到这一个 `kind`。**无侧分布**用 `zf.plot(kind="monitor", show_dist=False)`——这会切到裸的 `zf.LiveLive` (整条轨迹占满宽度、仍显最新值), 装载率这类右侧直方图/ σ 拟合无物理意义的量用它。**注意区分两个口径**: `LiveLive` 与 `LiveLiveDis` 在 Python 类层面是两个独立 `plot type` (后者继承前者、加分布带), 但 `plot(kind=...)` 与 Task 面板层**只有一个** `monitor kind`, 有无侧分布是 `show_dist` 这个开关决定的——没有 `kind="monitor\_nodist"` 这种写法 (写了会 `ValueError`)。

sites -> `zf.LiveSiteMap` 占据叠加图: `data_x` 是 `(N,2)` 站点中心 (相机像素), `data_y` 是逐站占据 (≥ 0.5 判有原子); 可选 `image` (相机帧底图)/`roi_radius`。无 `colormap` (二值空心圆环, 配色见 `SITE\_OCCUPANCY\_STYLE`)。

pulse -> `zf.PulseSequenceFigure` 静态脉冲时序图。

7.4 zf.panel_plot() 与尺寸预设 (仪表盘面板)

一句话用途: `plot()` 的仪表盘特化——选一个 `kind` + 一个 `PANEL_SIZES` 预设, 拿到尺寸正确、风格一致的实时面板 (不 `display`、不附 `DataFigure`, 由宿主嵌入 `canvas`)。

完整签名: `zf.panel_plot(data_x, data_y=None, *, kind, size="2x2", **kwargs)`

每个参数: `kind` (同 `plot`, 面板常用 `2d/sites/1d/monitor/hist`); `size` (见下枚举); `**kwargs` 为数据选项 (含 `relim\_mode`: `tight/normal/fixed`, `fixed` 钉住 `fixed\_lo/fixed\_hi`, 数据跳出也不动, 对 1D y 轴与 2D-Site map 的 `clim` 都生效); **几何不可配**。

尺寸枚举与辅助:

- `zf.PANEL_SIZES = ("1x2", "2x2", "4x2", "1x4", "2x4", "4x4", "4x8", "8x4", "8x8")`: 含义 = **行 × 列的“半单元”数**。2x2 = 标准 `plot` 区域 (数据区 480×360), 1x2 = 半高, 2x4 = 双宽, 4x4 = 双倍, 更大的 4x8/8x4/8x8 用于大网格。这份枚举以 **live.py** 的 `PANEL_SIZES` 为**单一来源**, 随其增删。
- `zf.panel_plot_spec(size="2x2", *, kind="default")` → `FigureSpec`: 返回该尺寸的几何规格 (所有面板类型共用同一组边距, 数据区随 `size` 预设缩放)。`kind` 是

keyword-only 的边距变体开关 (默认 "default"; 只有 kind="pulse" 加宽左边距给通道名留位)。

- `zf.panel_size_cells(size) → (rows, cols)`: 解析尺寸串; 非法尺寸报 `ValueError`。

示例: `panel = zf.panel_plot(frame, kind="2d", size="2x2", labels=...)`

不可配置的边界: 尺寸是唯一旋钮 (只能从 `PANEL_SIZES` 选); 所有面板共享同一 plot 区域和同一组边距, 故 `2x2` 主图恰好 `360×360` 方形、与 colorbar 顶底对齐, `2x2` = 两张 `1x2` + 间隙。普通用户用 Task 控制台 GUI 即可, 这几个是给**自定义宿主**程序化排版用的。

7.5 zf.show_pulse_gui() ——脉冲编辑器入口

一句话用途: 在独立 Fluent 窗口里打开脉冲编辑器; notebook 里编辑/上传脉冲的首选入口。

完整签名:

Python

```
zf.show_pulse_gui(*, state=None, channels=None, sequencer=None,
                  experiment=None, channel_labels=None, channel_pins=None,
                  scale=None, window_ratio=...) -> PulseSequenceEditor
```

每个参数: `state` (`PulseTableState`, `None`= 空白); `channels` (通道名序列); `sequencer` (传入的 `SequencerDevice`, On Pulse 用它上传——**frontend 不拥有硬件动作**); `experiment` (可选实验对象); `channel_labels/channel_pins` (显示名 / 物理引脚映射); `scale` (`None`= 自动, 见边界); `window_ratio` (窗口占屏比例)。

示例: `ed = zf.show_pulse_gui(sequencer=exp.devices.sequencer, channels=[...])`;
命令行入口是 `pulse\gui.bat`。逐控件操作见前面“逐个按钮”章。

不可配置的边界: `scale=None` 时走**唯一的 auto-scale 规则** (不要手动钉 `scale`, 否则破坏高 DPI 视觉一致性); 窗口几何/Fluent 样式由 frontend 拥有。

7.6 zf.show_task_console() ——Task 控制台入口

一句话用途: 打开可配置实时仪表盘窗口, 把面板接到 `SignalHub` 信号上。

完整签名:

Python

```
zf.show_task_console(*, hub, state=None, task=None, running_nodes=(),
                    measurements=(), processors=(), tasks=(),
                    ↪ session=None,
                    scale=None, window_ratio=0.84,
                    title="TaskConsole@Zou lab", on_close=None) ->
                    ↪ TaskConsole
```

每个参数: `hub` (必填, `SignalHub` 实例——数据总线) ; `state` (显式 `TaskConsoleState` 布局) ; `task` (**你存的布局:** `tasks/` 里的 JSON 名或一个 JSON 路径, **没有内置预设**; `state` 优先于 `task`) ; `running_nodes` (预挂的逻辑节点序列, 开窗时**自动 start**——已在跑的保留自己的速率; `TaskConsole._\__init__\__` 故意不 `start`, 留给测试/Notebook 确定性手动 `step()`) ; `measurements/processors/tasks` (**自动发现**的三份目录, 通常传 `exp.readout.measurement\_specs()/processor\_specs()/task\_specs()`——每个都**列进标题栏 Add Panel 下拉**的对应组, 选中 Add 成一个 Logic 节点; 为空则下拉只有 plot

kind) ; session (exp, 相机/标定等节点建器从它取设备; -config/虚拟启动都自动传) ; scale/window_ratio/title (窗口外观); on_close (关窗回调, 如 exp.close 一并 disconnect/safe 设备)。

示例:

Python

```
console = zf.show_task_console(hub=hub, session=exp,
                               measurements=exp.readout.measurement_specs(
                                   ↪ ),
                               processors=exp.readout.processor_specs(),
                               tasks=exp.readout.task_specs())
```

然后实验循环里 hub.publish(...) (见“Task 控制台”章接线示例), 或从 Add Panel 选 Temperature 建一个 Logic 节点、在它自己的 Edit 标签一键 Start, 再加 Plot 面板连它发布的信号。

不可配置的边界: 面板几何/卡片排版/吸附网格由 frontend 拥有; 可配的只有 kind + size + Source 表达式 + 声明式参数 + Setting 的基本显示项 (colormap/relim/unit)。

7.7 SignalHub ——仪表盘数据契约

导入: from Zou_lab_control.neutral_atom.core.signals import SignalHub (它是控制台的数据总线)。

发布侧 hub.publish(values, *, shot=None) → int: 每个值被**强制转成 float ndarray (标量->0 维) 并拷贝** (生产者可随意复用缓冲), 返回新 version。

消费侧: latest(name) (0 维->float, 否则 ndarray 拷贝; 不存在报 KeyError)、history(name, n=None) → ndarray (**形状 (shots, *value_shape)**, axis 0 是 shot; 形状变过只保留最近同形状连续段)、names() → list、snapshot_latest() → dict (一致快照 = 表达式命名空间)、属性 version/shot。

约定俗成的信号名 (沿用它们-> 同一命名任务在虚拟/真机间零修改通用): frame (相机帧)、counts、occupied (均 (repeat, n_sites) 块, repeat_mode=average 即逐站装载概率)、rate (本块装载率标量)、centers (站点中心 (N,2))、thresholds、shot。

示例: hub.publish({"counts": c, "rate": r, "shot": float(s)}), 再开一个 Rolling trace 面板把 Source 指到信号名即可。

7.8 Measurement 框架——一键可扫描的测量 (Logic 节点)

一句话用途: 把一个“扫一个被绑定的脉冲参数、每点采几帧、约简成一条曲线”的测量声明一次, **API 一行与 GUI Start 按钮同源**地跑它。这些类在 Zou_lab_control.neutral_atom.operations.measurement (声明 MeasurementSpec/ParamDecl) 与 ... operations.logic(ScannedMeasurementNode 包装); **目录入口**是子系统方法 exp.readout.measurement_specs()。

exp.readout.measurement_specs() → list[MeasurementSpec]: 返回 GUI 可驱动的测量目录 (内置温度 + 读出时长, 加上任何新发现/注册的)。**这个目录是自动发现的、不是写死的 list:** 它由 operations.measurement_registry 装配——operations/measurements/ 包里每个 @measurement 工厂 + notebook 里 na.register_measurement(...) 的。直接传给 show_task_console(measurements=...)。

加一个自己的测量 (自动被发现): 写一个 build(readout) → MeasurementSpec 工厂 (build 闭包用 readout.session 拿 exp、复用 readout.build_*_scan 或自己拼 ScannedMeasurement), 两条路任选其一让它进目录:(1) 把工厂放进 operations/measurements/你

的文件.py 用 `@measurement` 装饰——**丢个文件就被自动载入**，下次开 `task_console` 它就在 Add Panel 里；(2) 在 notebook 里 `na.register\_measurement(你的工厂)` (临时用 `na.unregister\_measurement` 撤销)。目录按 `order` (内置在前) 排序、按 `name` 去重 (后注册覆盖, 可借此覆盖内置)。下划线开头的模块被跳过 (留给包内私有助手)。

MeasurementSpec (声明式单一真相源): 字段 `name` (显示名)、`params` (`ParamDecl` 元组, 声明一次, API 默认与 GUI 控件都从它派生)、`result\_labels/x\_key/y\_key` (曲线轴名 + 发布到 hub 的信号名)、`build(**param\_values) → ScannedMeasurement` (返回**未跑**的测量)、`metadata` (如温度拟合的 `capture_radius` 换算)。辅助: `spec.defaults()` 给每个参数的默认值, `spec.param(key)` 取某个声明。逐站点结果的 2D 网格视图是对 `<y\_key>\_sites` 信号的 `reshape` 表达式 (用阱阵列的网格形状), 不是 `spec` 上的字段。

ParamDecl (一个参数的声明): `ParamDecl(key, label, kind, default, unit="", lo=0.0, hi=1e12, required=False, choices=(), tooltip="")`. `kind ∈ {float, int, axis\_range (值=(min,max,points), GUI 出三框), bool, choice}`。**任何值都不被 eval**——消费方按 `kind` 校验/强转 (confocal 的“不自由文本 eval”教训)。

ScannedMeasurementNode(把扫描测量包成 Logic 节点): `ScannedMeasurementNode(hub, measurement, *, x\_key, y\_key, prefix="")` (在 `operations.logic` 与 `CameraMeasurement` 同为 `LogicNode` 子类)。每个 `step()` 前进**一个扫描点**并 `publish` 累积曲线 `\{x\_key: x[:k], y\_key: y[:k], scan\_done\}` (仅逐站点 `reducer` 另发扁平向量 `<y\_key>\_sites`; **不发可派生量**——网格是 `reshape` 表达式, `shot` 计数走面板命名空间的全局 `hub.shot`)。**有限扫描自停**: 最后一点发布后置 `stop`, 后台 `start()` 线程自退; `finished` 报告完成, `run\_to\_completion()` 同步跑完剩余点 (测试/headless)。它只碰 `measure` 与 `hub.publish`, 零 `import` 具体后端、零读仿真真值 (虚拟 == 实机)。GUI 里一个 **Measurement: < 扫描名 >** Logic 节点 Start 时内部就是建一个这个 Node (**只 publish 到 hub、不自动出图**——`plot=False`, 你自己加 Plot 面板连它发布的信号)。

示例 (headless 跑一个温度扫描并喂 hub):

Python

```
from Zou_lab_control.neutral_atom.operations.logic import
↳ ScannedMeasurementNode
spec = next(s for s in exp.readout.measurement_specs() if "Temperature" in
↳ s.name)
meas = spec.build(t_off=(0.0, 300.0, 13), shots=16, per_site=False) #
↳ t_off 单位 us; capture_radius 不是采集参数, 它是 fit_temperature 的入参
# 节点按测量的 slug(spec.key)发布信号: temperature_t_off / temperature_survival.
node = ScannedMeasurementNode(hub, meas, x_key=spec.x_key,
↳ y_key=spec.y_key, prefix=f"{spec.key}_")
node.run_to_completion() # 或 node.start(rate_hz=...) 后台跑; 1D 面板 source
↳ = value = signal, picker 选 temperature_survival
```

notebook 里一行起同一测量: `exp.readout.temperature(...)` (见主手册)。**两条路调同一个建器, 不会漂移。**

7.9 PDF 渲染 API

zf.render_tex_pdf(tex, output_pdf, *, runs=2, xelatex=None, halt_on_error=True, assets=None) → Path (首选): 从 **TeX 串或.tex 路径**一次性出 PDF。在**临时目录**里 XeLaTeX 两遍, **只把最终 PDF 拷回**目标, 绝不在目标目录留 `.tex/.aux/.log/.toc/.sty`。`tex` 是字符串时, `assets=< 目录 >` 会被拷进临时构建树 (让 `assets/foo.png` 这类相对引用可用); 失败时只在 `output_pdf` 旁留一个 `.build.log`。

Python

```
from Zou_lab_control import frontend as zf
# 直接给字符串
zf.render_tex_pdf(r"\documentclass{article}\begin{document}你好\end{docume}
↪ nt}", "out.pdf")
# 或给一个 .tex 文件 (同目录 assets 自动随行)
zf.render_tex_pdf("notes/manual.tex", "notes/manual.pdf")
```

zf.render_notes_pdf(output_dir, *, filename, title, ..., body, compile_pdf=True, ...) → **NotesBuildResult (整本手册)** : 在内存里拼装标题页 + 目录 + 正文, 再经 **render_tex_pdf** 在临时目录编译, **output_dir** 里只留.pdf (外加它引用的 **assets/**)。本手册自身就是用这条路径生成的 (见 **frontend/notes.py** 的 **build_frontend_manual**)。 **write_notes_tex/compile_notes_pdf** 是**调试**用的就地写盘助手, 不在默认构建路径上。

7.10 一页速查 (“我想做 X -> 调用 Y”)

我想.....	调用
画一维/二维/直方图/脉冲/站点图	zf.plot(x, y, kind=...)
做仪表盘面板 (嵌进宿主)	zf.panel_plot(x, y, kind=, size=)
打开脉冲编辑器 GUI	zf.show_pulse_gui(sequencer=)
打开实时监控台	zf.show_task_console(hub=, task=)
打开设备管理器 GUI (编辑/换设备)	exp.device_manager() (会话绑定) 或 na.device_manager(config) (先建配置再 Init)
只读查看设备状态	exp.device_viewer() (监 控 台 顶 栏 Devices 按钮即此)
推送一个 shot 给监控台	hub.publish(...)
TeX/字符串 -> PDF	zf.render_tex_pdf(tex, out)

每一行的共同前提: **美术/几何/dpi** 由 **frontend** 拥有, 你给数据和语义选项, 拿回正确的图。

8

绘图 API

8.1 统一的数组契约

`Zou_lab_control.frontend.plot / live` 用同一套数组契约: `data_x` 是 $(N, \text{coord_dim})$, `data_y` 是 $(N, \text{channel_dim})$. `coord_dim = 1` 画一维线图, `coord_dim = 2` 画二维扫描图, `kind="hist"` 把 `data_x` 当作数值数组。未完成的数据用 `NaN` 表示, 前端可据此推断进度。

Python

```
import numpy as np
from Zou_lab_control.frontend import plot

x = np.linspace(737.0, 737.2, 220).reshape(-1, 1)
y = (lorentzian(x) + noise).reshape(-1, 1)
p = plot(x, y, labels=("Wavelength (nm)", "Counts/0.1s", "Counts"))
p.data_figure.lorent(is_display=True)      # 叠加洛伦兹拟合
```

8.2 直方图与阈值

Python

```
hist = plot(roi_counts, kind="hist", bins=55, thresholds=[45],
            labels=("ROI counts", "Shots", "Population"))
```

8.3 实时刷新

用 `update="watch"` 时, 返回对象启动一个前端定时器, 从共享数组刷新, 而采集代码在另一侧 in-place 修改这些数组。worker 不直接动 Matplotlib artist; 前端拥有 artist 和刷新。绘图刷新依赖事件循环, 因此阻塞式硬件采集不能直接跑在 UI 线程里。

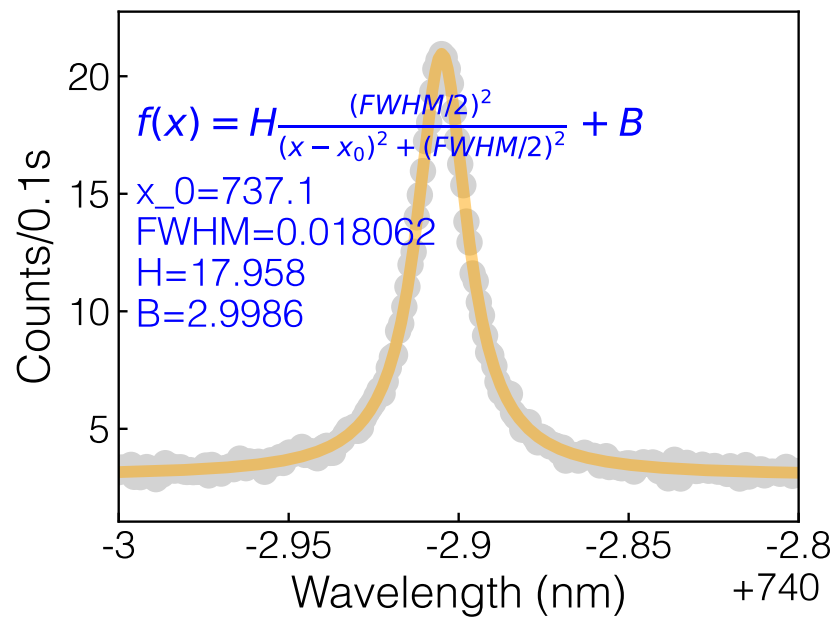


Figure 8.1: 一维线图加洛伦兹拟合。

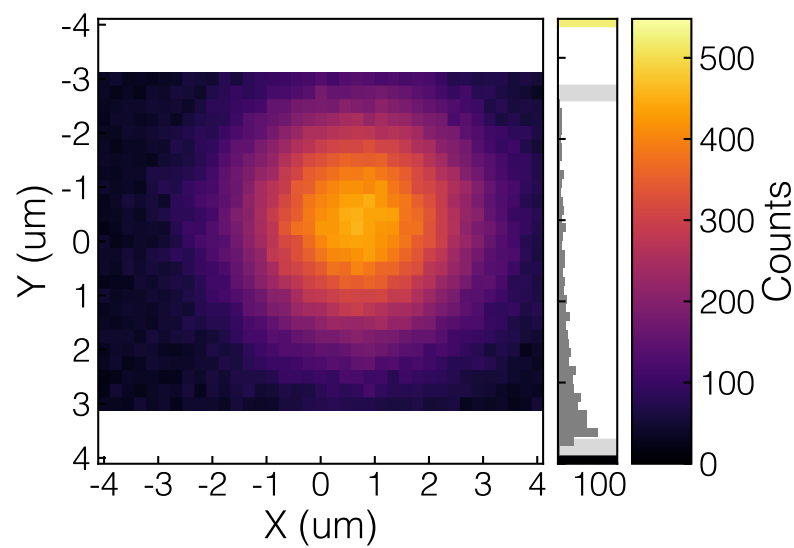


Figure 8.2: 二维扫描图: data_x 为 (N,2), data_y 为 (N,1)。

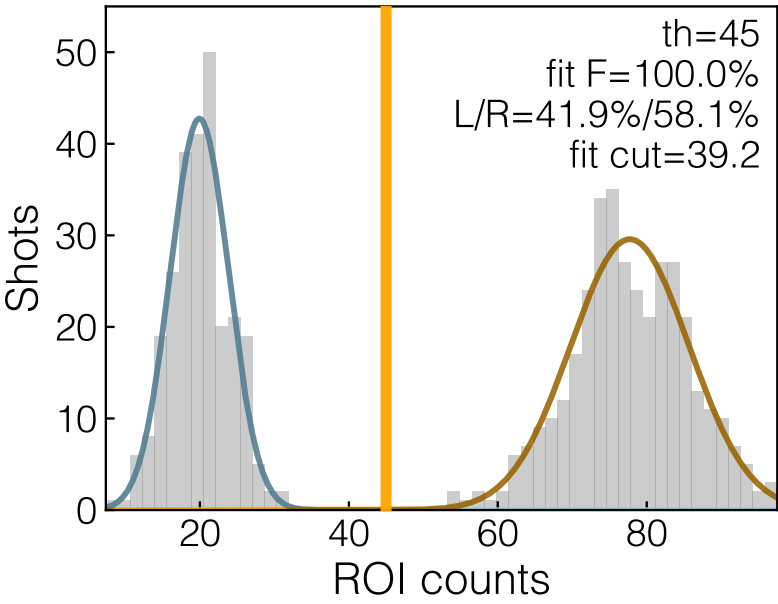


Figure 8.3: 带阈值的 ROI 计数直方图，用于亮/暗判定。

9

保存与回看图 (notebook 与 GUI 同一套核心)

9.1 一套核心，两种调用

把一张图存下来、以后再打开重看 / 重拟合 / 换个图型看，是一件事，不是两件。所以**存图的富捕获逻辑只有一份**，放在 frontend 之外的中立核心 `neutral\_atom.operations.figure\_capture` (`capture\_figure\_signals + capture\_figure\_provenance`，只依赖 SignalHub 与逻辑节点契约、不 import 任何 Qt/前端)。你有两种调用它的方式，**产生的 .npz 逐字一致**：

- **notebook**：任何绘图对象 (`plot(...)` 返回的 `plotter`，或它的 `DataFigure`) 都有 `.save(path)` 方法；
- **GUI**：Task 控制台面板的 **Save Fig** (在面板的 **Edit** 标签里)。

两条路走的是**同一个 figure_capture 核心**——不是两套实现。所以“notebook 里 `p.save()` 出来的图”和“在 GUI 面板里 Save 出来的图”是同一种文件、能用同一个查看器打开。

9.2 存一张图：p.save()

`.save(path)` 写**两个**文件：一张图片 (`png` / 可选 `pdf` / `jpg`) 和一个同名 `.npz` 数据包。npz 顶层永远是三个键 `data\_x` / `data\_y` / `info`——图型、单位、拟合、来源等都折进 `info`，所以**一个文件既能当图看、又能当数据重开**。

Python: 最简单的存图

```
import numpy as np
from Zou_lab_control import frontend as zf

x = np.linspace(737.0, 737.2, 220).reshape(-1, 1)
y = (lorentzian(x) + noise).reshape(-1, 1)
p = zf.plot(x, y, labels=("Wavelength (nm)", "Counts/0.1s", "Counts"),
    ↪ display=False)
p.data_figure.lorent(is_display=True)          # 叠加洛伦兹拟合
```

```
out = p.save("scans/probe_scan")          # 写
↪ scans/probe_scan_<时间戳>.png + .npz
# out = {"figure": <...>.png, "data": <...>.npz}
```

图片格式用 `image\_ext=` 选: `p.save("scans/probe", image\_ext="pdf")` 存 PDF。
npz 数据不随图片格式变——换成 pdf 或 jpg, 里面的数组 + `info` 完全一样。

9.3 富捕获: bind_source 把“数据从哪来”盖进去

上面的例子是**裸数组图**——只存下 `data\_x / data\_y` 和基础 `info`。当图是从一次**活实验**里长出来的(相机帧、逐站计数、一条扫描曲线), 你往往还想让存下来的文件记住“**这批数据是什么信号、采它时设备是什么状态**”。这就是**富捕获**, 由 `bind\_source` 打开:

Python: 绑定来源后再存 = 富 npz

```
# hub 是实验循环 publish 数据用的 SignalHub; node 是产出这批信号的逻辑节点
p = zf.plot(x, y, labels=("shot", "counts", ""), display=False)
p.bind_source(hub, node, inputs=["counts"],
              resolve_node=resolver, session=exp)  # 盖上来源戳 (返回 self,
              ↪ 可链式)
p.save("scans/loading")                          # 现在是“富 npz”
```

`bind\_source(hub, node, *, inputs, resolve\_node=None, session=None)`
 的五个参数:

hub 这批数据 publish 到的 `SignalHub`;

node 产出面板第一个输入信号的**逻辑节点** (measurement / processor);

inputs 这张图接的信号名列表 (如 `["counts"]`);

resolve_node (可选) 一个 **信号名** → **产出它的节点** 的映射器, 用于**向上游追设备**——一个 processor 面板 (自己不持设备) 据此把它源头 measurement 的相机 / sequencer 状态**提到顶层**;

session (可选) 没有节点产这批数据时的兜底设备快照来源。

绑定后 `.save()` 变成**富保存**, 往 `info` 多折两块:

- `info['signals']`——面板消费的**原始 hub 信号块** (按裸名 + 角色 value/x/centers/frame)。
Site map 靠它才能完整回来: 它的底垫相机帧和每站中心是从 flat `data\_x/data\_y` 里**恢复不出来的**, 必须原样存下;
- `info['provenance']`——采这批数据时的**设备状态记录** (camera + sequencer 快照 + 采集参数), **永远在顶层、格式统一**: 不管这张图直接接的是 measurement 还是它下游的 processor, 设备快照都提到同一层, 读的时候一致。

没绑定就优雅退化: 不调 `bind\_source` (一张纯数组图) 时, `.save()` 就只写基础的 figure + npz, 绝不报错。所以“随手 `p.save()`”永远能用, 绑定只是让存下来的文件更富。**这正是 GUI 面板 Save 在内部做的事**: 面板 Save 先 `bind\_source` 到该面板接的信号, 再调同一个 `DataFigure.save`——所以两边 npz 逐字一致。

9.4 回看: na.load_figure 与 exp.figure_viewer

存下来的 `.npz` 有两种打开方式, 都**不需要连硬件**:

(一) 代码里轻量查看——`na.load\_figure(npz)` 返回一个 `SavedFigure` 句柄:

Python: 无硬件重开一张存下的图

```
import Zou_lab_control.neutral_atom as na

sf = na.load_figure("scans/probe_scan_2026_06_30_14_02_11.npz")
print(sf.info_summary())          # 它存了什么:
↪ 名字/来源/图型/标签/单位/形状/进度/view/拟合
print(sf.compatible_kinds())      # 这份数据能被当成哪些图型看 (由数据形状推, 非写死)
df = sf.plot()                   # 按存下的图型重画, 复刻原图 -> 一个 DataFigure
df2 = sf.plot(kind="hist")        # 用另一种兼容图型重新解读同一份 data_x/data_y
df2.gaussian()                   # 重开的图照样能拟合
df2.save("scans/refit")           # 也能再存
```

`SavedFigure.plot(kind=None)` 是数据视角: 同一份 `data\_x/data\_y` 交给 `plot()` 工厂, 默认用存下的图型 (复刻原图), 换成 `compatible\_kinds()` 里的任意一个则用另一种 plotter 重新解读; 存下的 view 状态 (relim / 固定上下限 / cmap / 标签) 会种进重画的图, 你的 `overrides` 再覆盖它。

(二) GUI 里回看——`exp.figure\_viewer(path)` (或双击 `figure\_viewer.bat`、`zf.show\_figure\_viewer()`) 开一个纯查看器窗口。它是个真的、无采集的看图台:

左侧 Info 栏 分五个标签: **File** (文件本身)、**Plot** (名字/图型/标签/单位/存的 view/拟合)、**Measurement** (来源、`data\_x/data\_y` 形状、points/repeat、存下的信号块)、**Device** (provenance 里的采集时设备快照: camera + sequencer, 不管源是 measurement 还是 processor 都在顶层一致展开)、**Raw** (整个 info 字典原文)。

右侧 嵌一个真的 Task 控制台: 载入一张图 = 往它的 hub 里发一条 `fig\_value` 信号, 并自动开一张复刻原图的面板; 于是缩放/拖阈值线/换图型/再拟合/再存都是控制台面板原本就有的能力, 不必另造。面板可以拖到第二列; 视口窄时看图台按需加横向滚动、需要时扩展成多列。

Browse 选图对话框列出 png/jpg 图片和 npz; 选中一张图片会自动去载入它的同名 .npz——所以你按缩略图挑图, 数据自动跟上。

9.5 面板 Save 与 pulse 预览也走同一套

Task 控制台面板的 Edit 标签底部有 Save Fig: 一个 png/pdf/jpg 格式下拉 + 按钮, 导出当前面板的图 + 同名 npz (npz 数据不随格式变)。它内部就是先 `bind\_source` 到该面板接的信号、再调 `DataFigure.save`——和你在 notebook 里 `p.bind\_source(...).save()` 是同一条路。

脉冲编辑器的 Preview 页也有 Save Figure: 它把当前预览的时序波形包成一个通用 DataFigure 再 `.save()`, 所以存出来也是标准的图 + npz (图型标记 1d), 一样能在 `figure\_viewer` 里回看——只是脉冲预览本身信息较少 (没有信号块/设备 provenance), 回看时以波形为主。

10

PDF 渲染 API

10.1 render_tex_pdf

一次性从 TeX 串或 `.tex` 文件生成 PDF, 首选 `render_tex_pdf`。它在临时目录里编译 (XeLaTeX 两遍), 只把最终 PDF 拷回, 调用方不必管 `.aux/.log` 等辅助文件。

Python

```
from Zou_lab_control.frontend.notes import render_tex_pdf

render_tex_pdf("docs/main_manual/main_manual_zh.tex",
               "docs/main_manual/main_manual_zh.pdf")
```

输入是 `.tex` 文件时, 同目录的资产 (如 `assets/foo.pdf`) 会被拷进临时构建树, 但同名的旧输出 (`.pdf/.aux/.log/.build.log`) 不会被拷。构建失败时只在输出 PDF 旁留一个 `.build.log` 供诊断 (包括 XeLaTeX 缺失或路径错误的情况)。

10.2 notes 风格的整本手册

`render_notes_pdf(...)` 用同一个干净编译器 (默认 `compile_pdf=True`, 临时目录编译、只产 PDF), 自动套用本仓库的标题页与目录样式。本手册自身就是用这条路径生成的 (见 `Zou_lab_control/frontend/notes.py` 里的 `build_frontend_manual`)。

11

修改指南

11.1 我要加一个新 Fluent 控件

扩展 `qt_fluent.py`, 复用 `Metrics` token 和 `scaled_px()`, 不要在 GUI 模块里另造样式或缩放。新控件应能在 `FluentFormGrid` 里以 `Metrics.row_h()` 高度对齐。

11.2 我要给脉冲 GUI 加一行

任何新行、spacer、标题、开关文字或局部 margin, 都必须保证 `Channel Names`、`Delay \& Scan`、`period` 卡片、`repeat bracket` 这几块的顶部控制行仍然对齐。改完后加 object-level 断言: 同一 channel 的 raw 标签、延时框、第一个 period 勾选框的 y 坐标相等。

11.3 我要改截图 QA

截图前必须跑事件循环并等待; `show()` 或状态变化后不要立刻 `grab()`:

Python

```
editor.show()
editor.grab_screenshot(path, settle_ms=1000)          # 首选 helper
# 没有 helper 时:
# app.processEvents(); QTest.QTest.qWait(1000); app.processEvents();
↪ editor.grab().save(path)
```

离屏截图可能漏字, 所以截图只作视觉 QA; 按钮文字、几何、状态仍要用 object-level 测试断言。涉及可视化的改动必须用 `devtools.capture\_user\_view("console"/"editor", out\_dir)` 验收: 它在子进程里按 `QT\_SCALE\_FACTOR = 1.0/1.25/1.5` 三档各渲染一次完整窗口 (重现 Windows 显示缩放下用户真正看到的效果) 并保存截图——DPR=1 的单张截图通过**不算**通过。更多规则见 `docs/MAINTAINER_NOTES.md`。

11.4 我要改本手册

正文模板在 `Zou_lab_control/frontend/content/manual_templates/frontend_manual_zh.texbody`, 示例图由 `Zou_lab_control/frontend/content/manuals.py` 的 `generate_frontend_manual_figures` 生成。改完用 `build_frontend_manual()` 重建 PDF。